



江南大学
JIANGNAN UNIVERSITY

数据可视化大作业

赛道 1:「职数洞见」多变量招聘数据可视分析

学院 人工智能与计算机学院

成员 花政林 韦金妹

学号 1193210125 1193210106

班级 人工智能 2101 班

2024 年 6 月 22 日

目录

1 问题描述	5
2 数据处理	5
2.1 实验环境	5
2.2 数据格式分析	5
2.3 数据预处理	6
3 关键技术描述	8
3.1 问题一：量化评估职位差异度	8
3.1.1 K-means 聚类分析	8
3.1.2 交互式轮播柱状-折线图绘制	10
3.1.3 交互式轮播堆叠柱状图绘制	11
3.1.4 可视化结果	13
3.1.5 结果分析	14
3.2 问题二：从多角度设计并展示职位画像	15
3.2.1 关键技能分析词云图绘制	15
3.2.2 城市偏好双 Y 轴柱状-折线图绘制	15
3.2.3 薪酬待遇交互式轮播箱线图绘制	16
3.2.4 可视化结果	17
3.2.5 结果分析	18
3.3 问题三：建模并识别薪酬模式和潜在的薪酬差异	18
3.3.1 特征相关性分析与构建	19
3.3.2 随机森林建模	19
3.3.3 可视化结果	20
3.3.4 结果分析	21
3.4 问题四：挖掘和总结地域招聘活动画像	22
3.4.1 偏好职位与薪水分布交互式轮播柱状-折线图绘制	22
3.4.2 行业类别与薪水分布交互式轮播堆叠柱状-折线图绘制	25
3.4.3 K-means 聚类分析	27
3.4.4 PCA 降维	29
3.4.5 可视化结果	29
3.4.6 结果分析	30
3.5 问题五：行业发展动态与新兴职位识别	31
3.5.1 行业发展动态	31
3.5.2 急需人才的新兴职位	31
3.5.3 理由与实际观察	31
4 附录：核心代码	32
4.1 问题一核心代码	32
4.1.1 交互式轮播柱状-折线图绘制	32
4.1.2 交互式轮播柱状-折线图绘制	33

4.2	问题二核心代码	33
4.2.1	词云图绘制	33
4.2.2	交互式轮播堆叠柱状图绘制	34
4.3	问题三核心代码	34
4.4	问题四核心代码	34
4.4.1	交互式轮播柱状-折线图绘制	34
4.4.2	交互式轮播堆叠柱状-折线图绘制	35

摘要

赛道 1: II「职数洞见」多变量招聘数据可视分析

1193210125 花政林 1193210106 韦金妹

为企业深入理解招聘市场的动态，优化人才招聘和人力资源管理策略。借助 DataInsight 所收集的招聘市场数据，对其进行数据预处理完之后，利用先进的数据分析和可视化技术，对这些数据进行深入分析，设计并实现一套可视分析解决方案，帮助相关机构直观感知和理解行业和地域发展动态。

针对问题一：从行业类别、薪资待遇、经验要求等多维度属性量化评估职位差异度。我们首先对职位特征进行了特征提取，其次对职位进行了**基于肘部法改进的 K-means 聚类**，将相似职位聚类在一起，降低了计算成本。之后对相似的职位进行可视化评估，操作流程如图

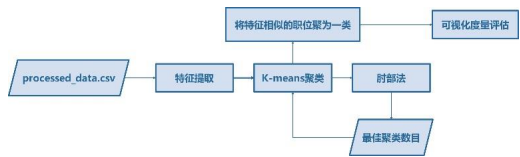


图 1 问题一流程图

图 1 所示。进一步地，我们将折线图与柱状图融合并采用交互可视化的方式绘制出交互式轮播柱状-折线图来研究评估不同职位之间的差异度。并从中得出行业集中度较低的职位薪资水平较高即需求量大的规律。

针对问题二：结合职位的关键特征挖掘，从多角度设计并展示职位画像。我们分别在**关键技能、偏好城市、薪酬待遇**三个角度进行**创新并设计交互式词云图、城市偏好双 Y 轴柱状-折线图以及薪酬待遇交互式轮播箱线图**，来展示职位的核心特征。并从中得出**关键技能需求**

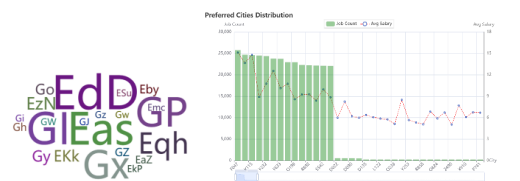


图 2 问题二流程图

市且通常职位需求量大的城市薪资水平较高。

针对问题三：不同的职位具有不同的薪酬待遇模式。请对薪酬待遇与职位、行业、地域等之间的潜在关系进行建模，识别薪酬模式和潜在的薪酬差异。我们首先选取了职位、行业类别、地域、经验以及学位五个特征并进行**特征量化与相关性分析**，之后，采用**随机森林算法**



分别用于识别薪酬模式的分类任务以及平均薪酬预测的回归任务，操作流程如图 2 所示。在训练集与测试集 4:1 的情况下，测得**分类模型的准确率高达 98.99%**，

回归任务的均方误差仅为 48.34。最后，根据得到的特征重要程度以此确定影响薪资的前 3 大主要因素以此作为**职位、经验要求以及行业类别**。

针对问题四：从多角度设计并展示招聘活动的地域特征，如偏好职位和行业类别等，并识别具有相似招聘特征的地域。我们使用**K-means 聚类**和**PCA 降维分析**，识别招聘特征相似的地域。可视化方面在全局与局部两个方面分别通过交互式轮播柱状-折线图展示各城市的**职位需求量和薪资水平**以及堆叠柱状-折线图展示行业类别分布和薪资水平。并从中得出具有相似招聘特征的地域。

针对问题五：基于上述分析结果，发现急需人才的新兴职位，并总结行业发展动态。我们结合前四问的实验结果以及实际，做出以下分析与建议：

- **行业类别与薪资：**行业集中度较低的职位薪资水平较高即人员的需求量较大。
- **经验与学位：**不同职位对经验和学历的需求差异显著。高学历和丰富经验的职位更容易获得高薪。
- **城市偏好：**可以通过适当调整职位的行业类别、薪酬分布以及经验要求等措施来平衡人才分布。

赛道 1: II「职数洞见」多变量招聘数据可视分析

1193210125 花政林 1193210106 韦金妹

一、问题描述

DataInsight 是一家领先的数据分析咨询公司,专注于为企业提供基于数据的洞察和决策支持。公司服务范围广泛,包括市场趋势分析、消费者行为研究、人力资源优化等多个领域。为了适应大数据时代的需求,提升在数据科学领域的专业服务能力,DataInsight 策划并推出了一项新的数据服务产品,旨在帮助企业深入理解招聘市场的动态,优化人才招聘和人力资源管理策略。为此,DataInsight 收集了招聘市场的数据,该数据集包含了四十万条详尽的招聘通知,覆盖多样化的职位类型和行业类别,并涵盖了广泛的行政区划。DataInsight 计划成立一个专门的市场招聘动态分析小组,利用先进的数据分析和可视化技术,对这些数据进行深入分析,以挖掘市场趋势,辅助人力资源优化。假如您是市场招聘动态分析小组的成员,请您设计并实现一套可视分析解决方案,帮助该机构直观感知和理解行业 and 地域发展动态,为企业人才招聘和求职决策提供可行的建议。请完成以下内容:

问题一: 分析职位招聘信息,从行业类别、薪资待遇、经验要求等多维度属性量化评估职位差异度。

问题二: 结合职位的关键特征挖掘,从多角度设计并展示职位画像,如关键技能、偏好城市、薪酬待遇等。

问题三: 不同的职位具有不同的薪酬待遇模式。请对薪酬待遇与职位、行业、地域等之间的潜在关系进行建模,识别薪酬模式和潜在的薪酬差异,利用图表的形式呈现结果并简要分析。

问题四: 挖掘和总结地域招聘活动画像,从多角度设计并展示招聘活动的地域特征,如偏好职位和行业类别等,并识别具有相似招聘特征的地域。

问题五: 结合上述分析结果,是否可以总结行业发展动态,并发现急需人才的新兴职位,并简要说明理由。

二、数据处理

2.1 实验环境

实验环境与工具如下:

- 编程语言: Python;
- 编译环境: Anaconda;
- 编译工具: Jupyter notebook。

2.2 数据格式分析

	job_title	city	salary	experience	education	company	company_type
0	e6f7b3df750c0155107f4aba490189caHn	K445	5-6K	EdD	GI	company_623498	type_BLfSmG
1	a3202d97cc6cef462c3e13473afdd30axs	K445	15-30K-15薪	EdD	Gx	company_728898	type_NKZQoO
2	02833b5e80d1bedec819210bc4b3ea26WV	K445	5-8K	EdD	GI	company_653468	type_BPRUkx
3	ad1ef9fc91583a99747340c810ba7eb2nV	K445	4-7K-13薪	EdD	GI	company_387472	type_NKZQoO
4	0d8ddc02b14ab0c4998cc33cb729b85ccn	K445	8-12K	Eqh	Gx	company_712558	type_QbeiZo

图 1: JobWanted 数据集预览

如图 1 所示, 本实验中所选赛道提供的数据集为一个模拟现实招聘市场的虚拟数据集, 该数据集包含了 430664 条详尽的招聘通知, 覆盖了 169540 种多样化的职位、267296 家不同企业、158 个行业类别, 并涵盖了 371 个行政区划。数据以 xlsx 格式提供。

数据集中的数据字段如表 1 所示:

表 1: JobWanted 数据集字段与信息格式

字段名称	job_title	city	salary	experience	education	company	company_type
字段含义	职位名称	行政区划	薪酬	经验要求	学历要求	企业	行业类别
数据类型	object	object	object	object	object	object	object

从表 1 可以看出, 所有字段的数据类型均为 object, 即字符串类型。此外, 在该数据集中, job_title、city、experience、education、company 以及 company_type 均采用虚拟代码的形式呈现。需要注意的是, 职位 (job_title) 与行业类别 (company_type) 之间存在多对多的关系。

2.3 数据预处理

对原始数据集进行缺失值处理、异常值处理以及标签转化:

(1) 缺失值处理

利用 Python 中 Pandas 库中的 “isnull().sum()” 方法来统计表格中每一行是否有缺失值, 经过统计发现原始数据中没有雀氏值, 统计输出结果如图 2 所示:

```
Out[2]: job_title      0
        city          0
        salary        0
        experience    0
        education     0
        company       0
        company_type  0
        dtype: int64
```

图 2: 缺失值统计结果

(2) 异常值处理

1) 重复值处理

根据题目条件分析, 当职位名称、公司类型、企业、城市相同时, 这些记录即为重复行, 即噪声数据。因为同一城市中同一公司的同一职位在经验、学历以及薪水分配上的做法应当相同。由于剩下的字段是经验和学历要求 (非数值) 以及薪水问题, 因此无法对这些数据进行有效聚合。考虑到重复数据仅占 367 条, 为防止噪声数据干扰, 对重复值的处理方式直接删除。重复值处理部分的 Python 核心代码如下:

```
1 num_duplicates = data.duplicated(subset=["job_title", "city", "company", "company_type"]).sum() # 统计重复行
2 data.drop_duplicates(subset=["job_title", "city", "company", "company_type"], inplace=True) # 获取所有重复的行
```

2) 无效值处理

根据题目要求, 当 **salary** 为“面议”时被视为无效薪水数据, 由于其占比很小仅有 13 条, 故为了防止其产生的噪声干扰, 将其直接去除即可。Python 核心代码如下:

```
1 data = data[data['salary'] != '面议']
```

(3) 薪酬数据处理

原始数据中, 薪酬包括“按月分配”、“按周分配”、“按天分配”、“按小时分配”以及“按单分配”等多种分配方式。为了将薪酬统一处理、增强数据的可比性, 结合实际情况将所有薪资数据都被标准化为月薪 (以 K 为单位), 并添加将原来的 salary 转化为最高薪水 (max_salary)、最低薪水 (min_salary) 以及平均薪水 (avg_salary) 三列, 转化方式如下所示:

- 按月分配:

- 带薪分配: 将月薪 $\times 12$ / 带薪月数, 如: 6K·13 薪, 则每个月的月薪平均为 $6K \times 12 / 13 = 5.54K$;
- 不带薪分配: 直接使用给定的月薪数据即可。

- 按周分配: 将周薪 $\times 4.33 / 1000$ (一个月平均 4.33 周);
- 按天分配: 将日薪 $\times 22 / 1000$ (一个月平均 22 个工作日);
- 按小时分配: 将时薪 $\times 22 \times 8 / 1000$ (每天平均工作 8 小时);
- 按单分配: 将单次薪酬 $\times 200 / 1000$ (假设每月可以处理 200 单)。

处理后的部分数据展示如图 3 所示:

job_title	city	salary	experience	education	company	company_type	salary_category	min_salary	max_salary	avg_salary
e6f7b3df750c0155107f4aba490189cahn	K445	5-6K	EdD	GI	company_623498	type_BLIsmG	按月结薪	5.00	6.00	5.50
a3202d97cc6cef462c3e13473afdd30axs	K445	15-30K·15薪	EdD	Gx	company_728898	type_NKZQoO	按月结薪	18.75	37.50	28.12
2833b5e80d1bedec819210bc4b3ea26WV	K445	5-8K	EdD	GI	company_653468	type_BPRUlx	按月结薪	5.00	8.00	6.50
ad1ef9fc91583a99747340c810ba7eb2nV	K445	4-7K·13薪	EdD	GI	company_387472	type_NKZQoO	按月结薪	4.33	7.58	5.96
0d8ddc02b14ab0c4998cc33cb729b85ccn	K445	8-12K	Eqh	Gx	company_712558	type_QbeiZo	按月结薪	8.00	12.00	10.00

图 3: 处理后的部分数据展示

在图 3 中, 新添加的 salary_category 列表示薪酬分配的方式, max_salary、min_salary 以及 avg_salary 表示该行数据所表示职位的最高薪酬、最低薪酬以及平均薪酬, 以 K 为单位。

(4) 虚拟代码转化

考虑到 job_title 列中的虚拟代码长度过长可能会影响到后续的可视化效果, 现 job_title 在原始数据表格中出现的先后顺序来赋予一个唯一的短标签来代替原始的虚拟代码, Python 核心代码如下:

```
1 job_titles = data_process['job_title'].unique() # 获取所有唯一的职位名称
2 job_title_mapping = {job_title: f"j{i+1}" for i, job_title in enumerate(job_titles)} # 创建职位名称映射表
```

最后, 选取处理后的数据表中的 job_title, city, experience, education, company, company_type, min_salary, avg_salary, max_salary, salary_category 列作为特征列, 构建新的数据表格 “processed_data.csv”, 数据表格部分数据展示如图 4 所示:

	job_title	city	experience	education	company	company_type	min_salary	avg_salary	max_salary	salary_category
0	j1	K445	EdD	GI	company_623498	type_BLfSmG	5.00	5.50	6.00	按月结算
1	j2	K445	EdD	Gx	company_728898	type_NKZQoO	18.75	28.12	37.50	按月结算
2	j3	K445	EdD	GI	company_653468	type_BPRUkx	5.00	6.50	8.00	按月结算
3	j4	K445	EdD	GI	company_387472	type_NKZQoO	4.33	5.96	7.58	按月结算
4	j5	K445	Eqh	Gx	company_712558	type_QbeiZo	8.00	10.00	12.00	按月结算

图 4: processed_data.csv 部分数据展示

三、关键技术描述

3.1 问题一：量化评估职位差异度

由于原始数据中包含 169,540 个不同的职位，逐一可视化这些职位将导致高昂的计算成本。考虑到不同职位之间存在大量相似性，在本问题的解决方案中，我们采用 **K-means 聚类** 的方法，将相似的职位聚类在一起，从而减少处理的职位数量，**聚类数目为通过肘部法获取的最佳聚类数目**。这样可以降低计算成本，并且使得后续的可视化分析更加简洁和高效。问题一的流程图如图 5 所示：

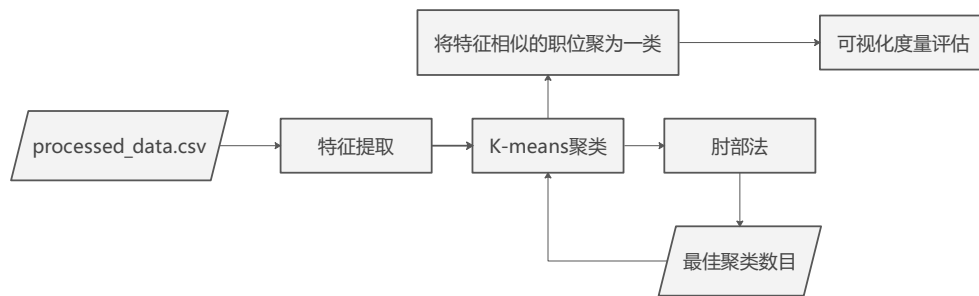


图 5: 问题一流程图

3.1.1 K-means 聚类分析

(1) 特征提取

1) 特征选择与量化

	city_encoded	experience_encoded	education_encoded	company_type_encoded	salary_category_encoded
0	0	0	0	0	0
1	0	0	1	1	0
2	0	0	0	2	0
3	0	0	0	1	0
4	0	1	1	3	0
...	...	...	...	...	...
430279	4	0	0	17	0
430280	4	0	0	32	0
430281	4	0	0	71	0
430282	4	0	2	131	0
430283	4	0	3	0	0

430284 rows × 5 columns

图 6: 特征选择与量化

对于每个职业 job_title 选取 city、company_type、avg_salary、experience 以及 education 作为其特征。进一步地，对 city、experience、education 以及 company_type 进行编码量化，所采用的量化方式为将每个属性字符串映射为一个唯一的整数。量化后的部分属性展示如上图 6 所示。

2) 特征构建

合并 job_title 相同的组并对组内每个属性取平均值。此外，为了提高聚类效果的准确性，将 city、company_type、avg_salary、experience 以及 education 特征通过 Python 中的 MinMaxScaler 函数进行归一化处理即得到所需的特征表格，部分数据如图 7 所示：

	city_encoded	experience_encoded	education_encoded	company_type_encoded	avg_salary	job_title
0	0.000000	0.000000	0.000000	0.000000	0.010202	j1
1	0.002703	0.111111	0.090909	0.038462	0.027927	j10
2	0.000000	0.000000	0.181818	0.051282	0.006265	j100
3	0.010811	0.000000	0.000000	0.006410	0.015107	j1000
4	0.202703	0.111111	0.090909	0.038462	0.025234	j10000
...	...	...	...	...	...	...
169529	0.691892	0.222222	0.090909	0.326923	0.018050	j99995
169530	0.691892	0.222222	0.090909	0.038462	0.030057	j99996
169531	0.691892	0.222222	0.000000	0.250000	0.009221	j99997
169532	0.691892	0.111111	0.090909	0.506410	0.033431	j99998
169533	0.691892	0.222222	0.000000	0.038462	0.008986	j99999

169534 rows x 6 columns

图 7: 特征构建

在图 7 中，每一行代表某个职位的特征向量。

(2) K-means 聚类分析

1) 算法思路

i. **初始化**: 定义一个空列表 wcss 来存储不同聚类数量下的总的簇内误差平方和 (WCSS), WCSS 计算方法如式 1 所示:

$$WCSS = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2 \quad (1)$$

其中， K 为聚类数量， C_k 表示第 k 个聚类中的数据点集合， x_i 表示第 i 个数据点， μ_k 表示第 k 个聚类的质心。WCSS 计算的是每个数据点到其所属簇的质心的欧几里得距离的平方和。是衡量聚类效果的一个指标，它反映了簇内数据点的紧密程度。**WCSS 值越小，表示簇内的数据点越接近质心，簇越紧密。**

ii. **循环计算 WCSS**: 在一个循环中，分别使用不同的聚类数量 (100 到 1000，每次增加 200) 进行 K-means 聚类，并计算对应的 WCSS 值，将其存储在 wcss 列表中。

iii. **绘制肘部图**: 使用 Matplotlib 绘制肘部图，以聚类数量为横轴，WCSS 为纵轴，通过观察图中的“肘部”位置来选择最佳的聚类数量。算法流程如下:

Algorithm 1 Elbow Method for Optimal Clusters

```

1: Initialize an empty list wcss
2: for i in range(100, 1000, 200) do
3:   kmeans = KMeans(n_clusters = i, random_state = 42)
4:   kmeans.fit(normalized_feature)
5:   Append kmeans.inertia_ to wcss
6: end for

```

2) Python 核心代码

```

1 from sklearn.cluster import KMeans
2 # 使用肘部法选择最佳聚类数量
3 wcss = []
4 for i in range(100, 1000, 200):
5     kmeans = KMeans(n_clusters=i, random_state=42)
6     kmeans.fit(normalized_data[['city_encoded', 'experience_encoded', 'education_encoded', '
7         company_type_encoded', 'avg_salary']])
8     wcss.append(kmeans.inertia_)

```

代码所绘制的肘部图如图 33 所示：

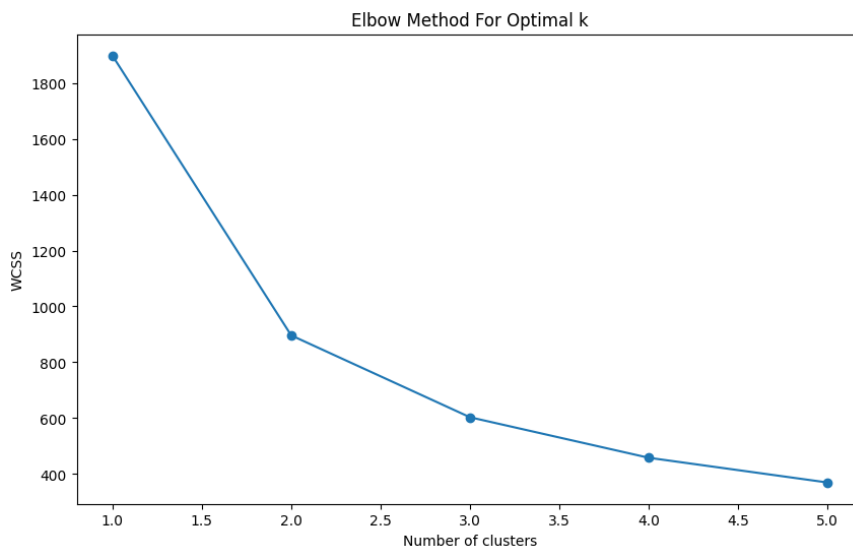


图 8: 肘部图

观察图 33 可以看出，“肘部”即折线斜率变化较大的位置在聚类数目为 1600 处，因此，**最佳聚类数目为 1600**。故取 $K=1600$ 时，重新进行 K-means 聚类，为每个 *job_title* 添加一个 *cluster* 标签以便后续可视化分析。

3.1.2 交互式轮播柱状-折线图绘制

利用 *pyecharts*，并融合柱状图与折线图来展示每个相似的职业即（聚类）中行业类别的数量和平均薪资统计数据。计算了每个聚类中的行业类别数量和平均薪资，并将这些信息合并到一个数据框中，然后通过条形图显示每个聚类的公司数量，通过折线图显示每个聚类的平均薪资，最终将两者结合在一起进行可视化展示。具体实现方法如下：

(1) 计算每个聚类中的行业类别的数量: 使用 `groupby` 方法按 `cluster` 分组, 并计算每个聚类中不同 `company_type` 的数量。将结果重命名为 `cluster` 和 `company_count` 列。

(2) 合并数据: 将行业类别的数量和薪资统计数据按 `cluster` 合并到一个数据框中 `cluster_data`。按聚类排序: 对 `cluster_data` 按 `cluster` 列进行排序, 以便可视化时横坐标按聚类顺序显示。

(3) 计算行业类别数量的平均值: 计算所有聚类中行业类别数量的平均值 `average_company_type_count`, 用于在条形图中添加平均线。

(4) 创建薪资统计的折线图与条形图: 使用 `pyecharts` 中的 `Line` 类创建折线图, 显示每个聚类的平均薪资。使用 `pyecharts` 中的 `Bar` 类创建条形图, 显示每个聚类的公司数量。设置条形图的透明度和颜色。并添加平均公司数量的标线。

(5) 将折线图和条形图组合: 使用 `bar.overlap(line)` 方法将折线图和条形图组合在一起。并设置图表的标题、坐标轴标签、轮播选项和图例位置。

核心代码详见附录, 交互式轮播柱状-折线图预览如图 9 所示:

Company Count by Cluster with Salary Stats

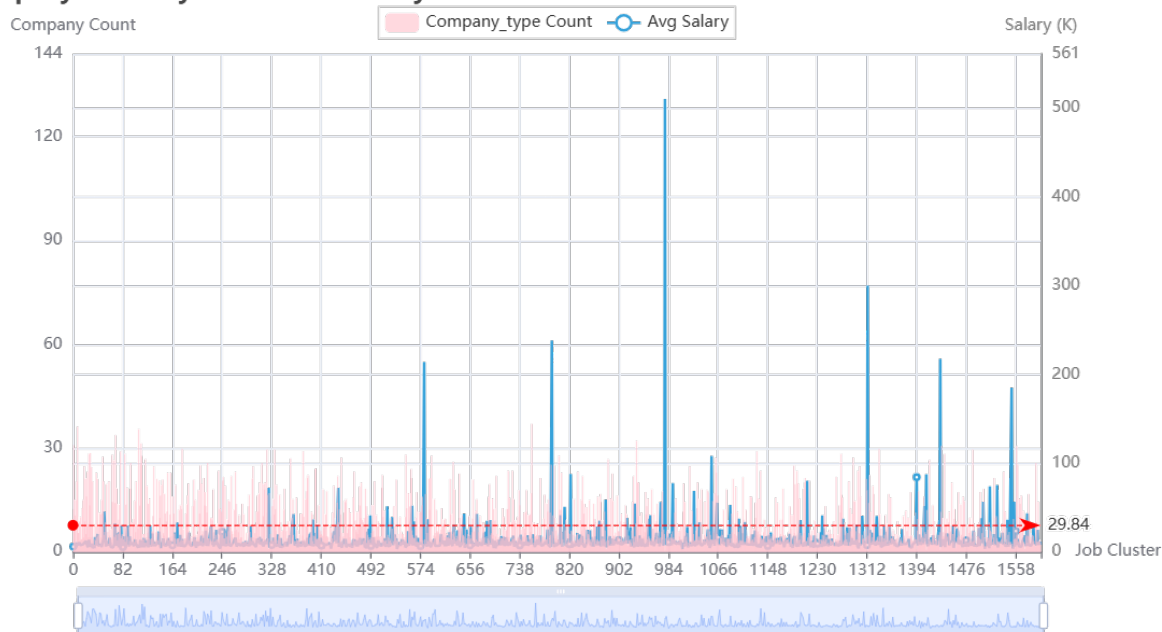


图 9: 交互式轮播柱状-折线图绘制预览

图 9 中可以看出, 不同职业在所属的行业类别的数量以及平均薪酬存在一定的差异。

3.1.3 交互式轮播堆叠柱状图绘制

利用 `pyecharts`, 分别绘制用于显示不同职位中职位数量与经验要求/学历要求分布的堆叠柱状图, 具体实现方法如下:

(1) 计算每个聚类的职位数量: 使用 `value_counts` 函数计算每个职位中的职位数量, 并将结果存储在 `cluster_counts` 数据框中。将 `cluster_counts` 列名设置为 `cluster` 和 `job_count`。

(2) 合并数据: 将 `cluster_counts` 和 `salary_stats` 数据框按 `cluster` 列合并, 生成新的数据框 `cluster_data`。对 `cluster_data` 按 `cluster` 列进行排序, 以便可视化时横坐标按聚类顺序显示。

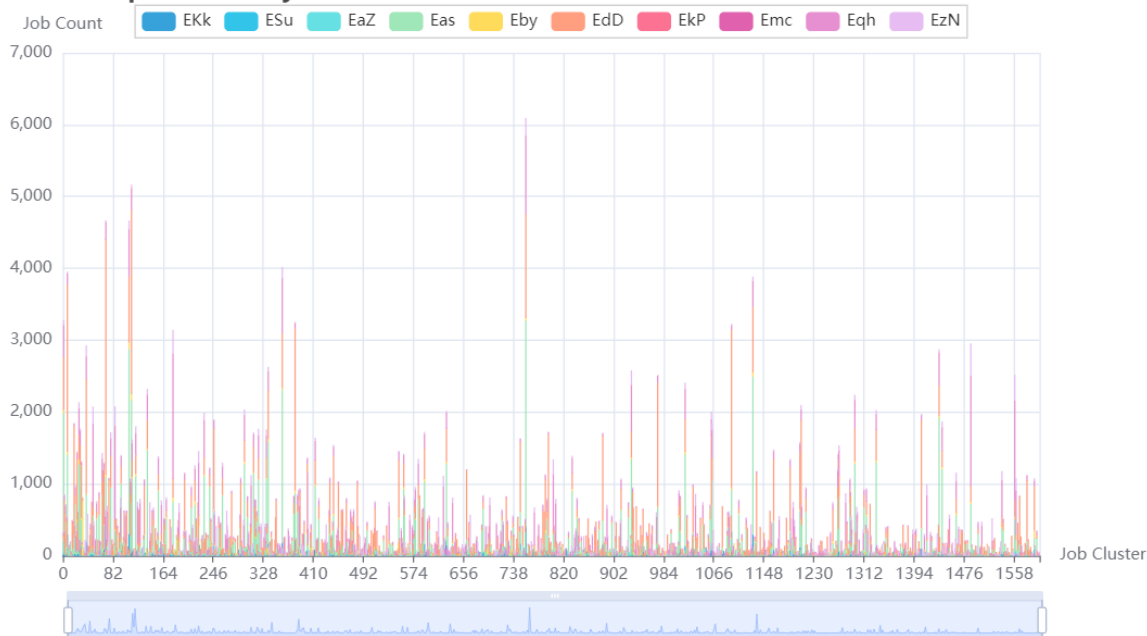
(3) 计算经验要求/学历要求分布: 按 `cluster` 和 `experience/education` 分组, 计算每个聚类中不同经验要求的职位数量, 并使用 `unstack` 将结果转换为透视表格式 `experience_distribution`。

(4) 创建堆叠柱状图：使用 `pyecharts` 中的 `Bar` 类创建柱状图，并设置初始主题为 `LIGHT`。使用 `add_axis` 方法添加 X 轴数据，数据为 `cluster_data` 中 `cluster` 列的值，转换为字符串格式。添加各个经验要求/学历要求的堆叠数据。

(5) 设置全局选项：设置图表的标题、X 轴标签、Y 轴标签、缩放选项和图例位置。

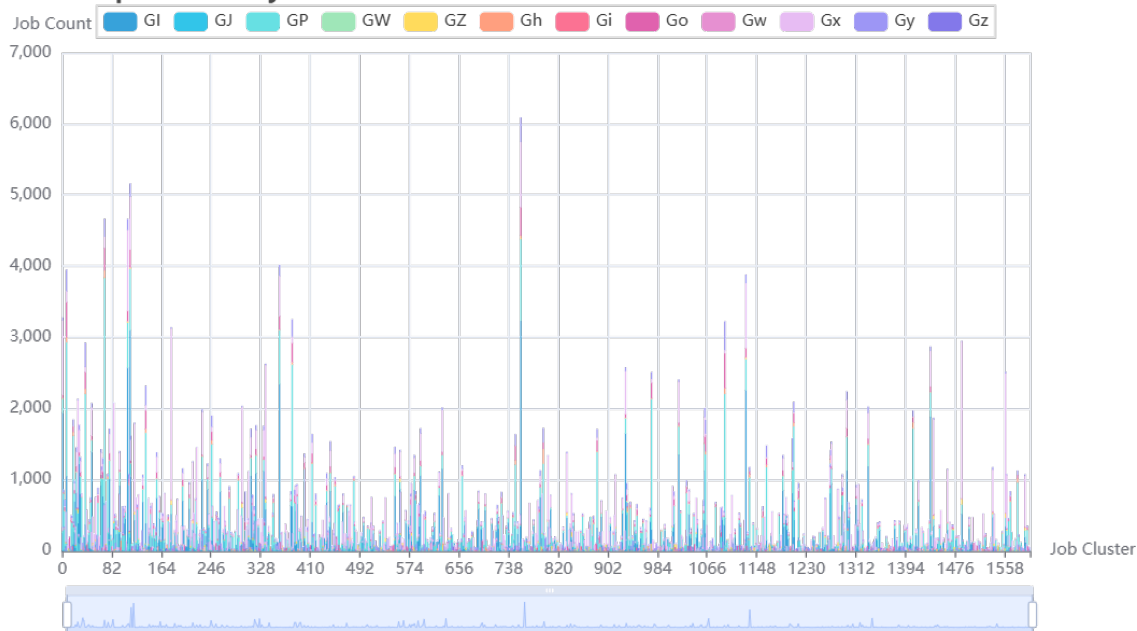
核心代码详见附录，交互式轮播堆叠柱状图预览如图 10 所示：

Experience Requirements by Job Cluster



(a) 经验要求堆叠柱状图

Education Requirements by Job Cluster



(b) 学历要求堆叠柱状图

图 10: 交互式轮播堆叠柱状图绘制预览

从图 10 中可以看出，不同职业对经验以及学位的要求有一定的差异。

3.1.4 可视化结果

(1) 交互式轮播柱状-折线图

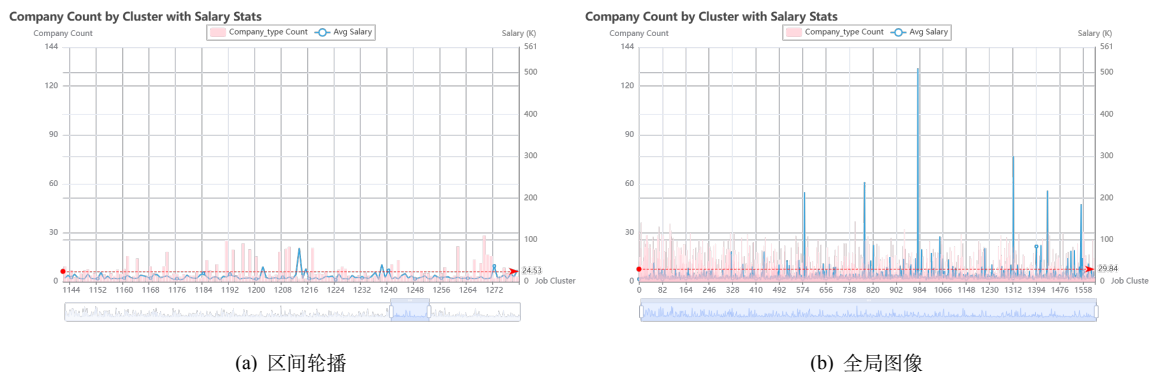


图 11: 交互式轮播柱状-折线图

在图 11 中，横轴为聚类，左侧纵轴为所属行业类别数量，右侧纵轴为平均月薪水平（单位为 K），其中每个聚类代表一类相近的职业。观察全图图像不难发现，不同的职位在所属的行业类别数量以及月薪水平上存在一定差异；同时，轮播每个区间可以发现，所属行业类别数量较少的职业往往薪资水平更高。

(2) 交互式轮播堆叠柱状图（经验要求）



图 12: 交互式轮播堆叠柱状图（经验要求）

在图 12 中，横轴为聚类，纵轴为所属行业类别数量，每个堆叠的柱子表示该职业经验要求的占比。观察全图图像不难发现，不同的职位在经验要求上存在一定差异，但是在大多数情况下，它们对 EdD 经验要求占较大比重；此外，观察区间轮播可以发现，当职位所属的行业类别较少时，Ekk 经验要求开始出现，EdD 占比下降，结合前面所属行业类别数量较少的职业往往薪资水平更高这一特点，而是 Ekk 经验可能与高薪有关。

(3) 交互式轮播堆叠柱状图（学位要求）



图 13: 交互式轮播堆叠柱状图 (学位要求)

在图 13 中，横轴为聚类，纵轴为所属行业类别数量，每个堆叠的柱子表示该职业学位要求的占比。观察全图图像不难发现，不同的职位在学位要求上存在一定差异，但是在大多数情况下，它们对 GP、GI 学位要求占较大比重；此外，观察区间轮播可以发现，当职位所属的行业类别较少时，Gz、Gh 学位要求开始出现，结合前面所属行业类别数量较少的职业往往薪资水平更高这一特点，而是 Gz、Gh 经验可能与高薪有关。

3.1.5 结果分析

通过分析三个交互式轮播图表（柱状-折线图、经验要求堆叠柱状图、学位要求堆叠柱状图），我们从行业类别、薪资待遇、经验要求和学位要求等多维度对职位差异进行量化评估。

(1) 行业类别与薪资待遇

1) 行业类别分析：在第一个图表中，通过横轴展示不同的聚类，我们观察到职位所属行业类别数量对薪资水平有显著影响。具体来说，行业类别数量较少的职业往往具有更高的薪资水平，这可能表明这些行业更为专业化或市场需求更高。

2) 薪资差异分析：薪资待遇的数据显示，不同聚类中的职业薪资水平存在显著差异。通过折线图和柱状图的结合，我们可以清楚地看到聚类与平均月薪之间的关系。

(2) 经验要求

第二个图表显示了不同聚类中职位的经验要求。显著的发现是，经验需求 Ekk 通常出现在行业类别较少的聚类中，这意味着 Ekk 更可能关联高薪职位。

(3) 学位要求

第三个图表聚焦于学位要求，我们发现大多数职位倾向于要求 GP 和 GI 学位，而较少出现的 Gz 和 Gh 学位要求职位往往集中在行业类别较少的高薪聚类。这意味着某些学位（如 Gz、Gh）更可能关联高薪职位，这可能反映了这些学位背后的高技能和专业性需求。

(4) 综合分析

结合所有视图，我们可以得出以下结论：

- **行业集中度与薪资：**行业集中度较低的职业往往拥有更高的薪资水平，反映了特定技能和专业知识的市场价值。
- **经验与学位的价值：**不同职位对经验需求以及学位需求有着较大的差异。大多数职位倾向于要求 GP 和 GI 学位，而较少出现的 Gz 和 Gh 学位要求、以及 Ekk 经验要求往往集中在行业类别较少的高薪职位。这意味着某些学位（如 Gz、Gh）、经验（如 Ekk）更可能关联高薪职位，这可能反映了这些学位背后的高技能和专业性需求。

3.2 问题二：从多角度设计并展示职位画像

3.2.1 关键技能分析词云图绘制

利用 Pyecharts，绘制所有职位的经验要求以及学位要求词云图展示职位当中需要对多的经验以及学位类别，具体实现过程如下：

(1) 合并经验和教育列为关键技能：将 `filtered_data` 数据框中的 `experience` 和 `education` 列进行合并，作为新的 `key_skills` 列。这种合并操作将每个职位的经验和教育要求组合在一起，**形成一个新的字符串，代表该职位的关键技能。**

(2) 将所有关键技能合并成一个字符串：将 `key_skills` 列中的所有字符串连接成一个大的字符串，以便后续进行词频统计。

(3) 统计词频：使用 `Counter` 类对合并后的大字符串进行词频统计。词频统计的结果是一个字典，键是单词，值是该单词出现的次数。并转换为适合 `pyecharts` 的格式。

(4) 使用 `pyecharts` 创建词云图，展示关键技能的词频分布。

核心代码详见附录，关键技能的词云图如图 14 所示：

Key Skills For Jobs



图 14: 词云图

从图 14 中可以看出，关键技能为具备经验 **EdD** 以及至少达到 **GI** 学位。

3.2.2 城市偏好双 Y 轴柱状-折线图绘制

为了综合考虑薪资水平以及职位数量来展示职位的偏好城市，本实验采用一种双 Y 轴柱状-折线图绘制方法来展示。具体绘制过程如下：

首先，从 `filtered_data` 数据集中统计每个城市的职位数量。其次，计算每个城市的平均薪水，将职位数量和平均薪水数据合并到一个数据框中。之后，绘制柱状图即使用 `pyecharts` 库中的 `Bar` 类创建柱状图，显示每个城市的职位数量。绘制折线图即使用 `pyecharts` 库中的 `Line` 类创建折线图，显示每个城市的平均薪水。最后，组合图表并设置全局选项。

核心代码详见附录，城市偏好双 Y 轴柱状-折线图预览如图 15 所示：

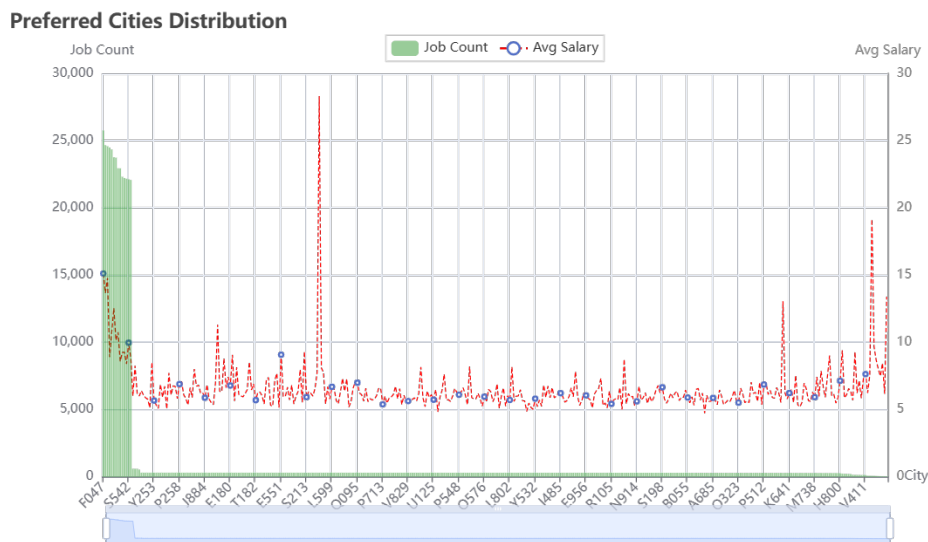


图 15: 城市偏好双 Y 轴柱状-折线图

从图 15 中不难发现，一般情况下，城市中职位数量较多的城市往往薪资水平比较高。

3.2.3 薪酬待遇交互式轮播箱线图绘制

为了展示职位的薪酬待遇，利用 Pyecharts 实现具有交互式轮播功能的箱线图，用于展示每个聚类（Cluster）的平均薪资（Avg Salary）分布情况，具体实现过程如下：

首先，从 filtered_data 数据集中提取每个聚类的平均薪资数据，并确保每个聚类至少有四个数据点。其次，将薪资数据转换为适合 pyecharts 的格式，以便生成箱线图。之后，使用 pyecharts 创建箱线图，展示每个聚类的薪资分布情况。最后设置图表的标题、图例位置、X 轴和 Y 轴标签，以及数据缩放功能，使图表更加美观和交互。

核心代码详见附录，薪酬待遇交互式轮播箱线图预览如图 16 所示：

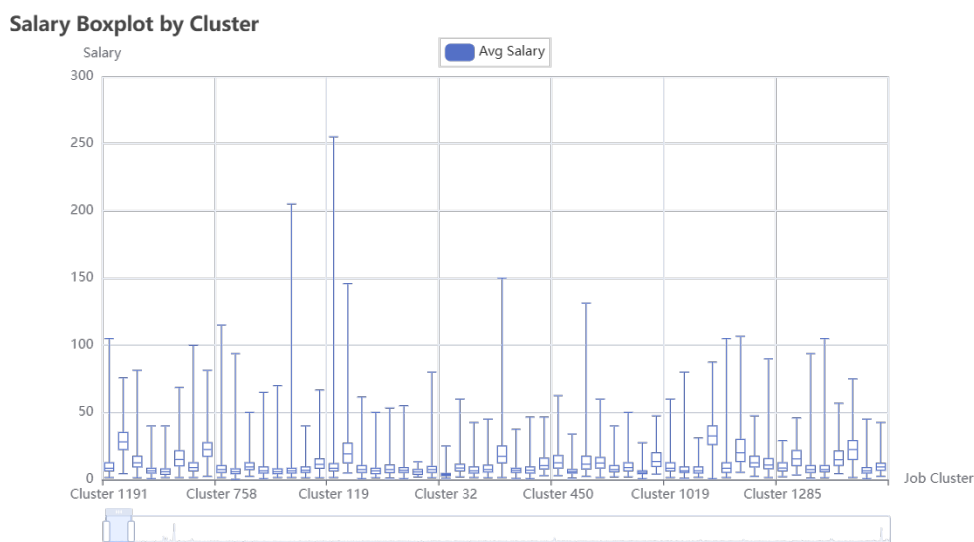


图 16: 薪酬待遇交互式轮播箱线图

从图 16 中可以通过轮播条来查看指定区间内的薪酬箱线图。

3.2.4 可视化结果

(1) 关键技能分析词云图绘制

Key Skills For Jobs



图 17: 关键技能分析词云图绘制

词云图中词的大小表示其频率或重要性。出现频率越高的词会显示得越大。从图 17 中可以看出，**EdD**、**GP**、**GI**、**Eas** 等词出现频率较高，因此这些技能在职位中被广泛需求。相对较小的词如 **Gx**、**Egh**、**Go** 等则表示出现频率较低，但仍然是职位中需要的技能。

(2) 城市偏好双 Y 轴柱状-折线图绘制

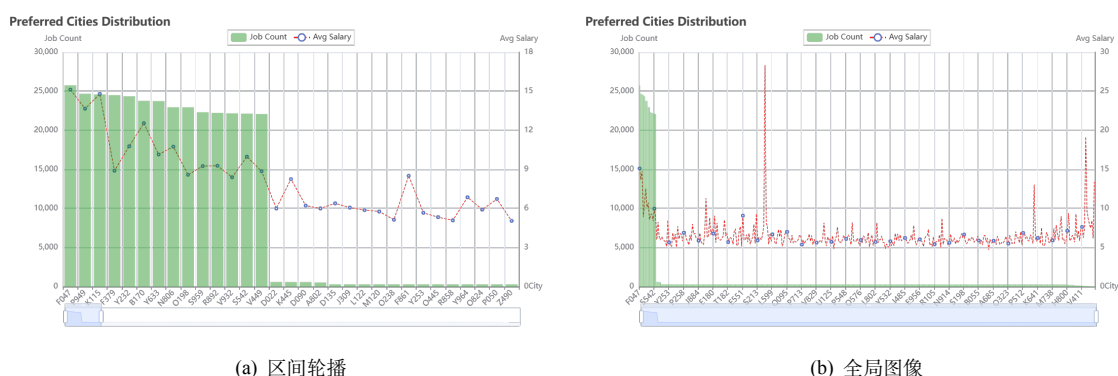


图 18: 城市偏好双 Y 轴柱状-折线图

在图 18 中，展示了不同城市的职位数量和平均薪水，通过柱状图和折线图的结合，提供了对城市职位市场的全面了解。

从右图可以看出，某些城市的职位数量明显高于其他城市。例如，图表的前几个城市职位数量都超过了 20,000，而后续大部分城市职位数量明显较低，表现出城市之间职位需求的差异。

平均薪水用红色虚线折线图表示。从图表中可以看出，职位数量多的城市，平均薪水并不一定高。但一般情况下，相较于职位数量少的城市会相对高一点。此外，在某些城市，虽然职位数量较少，但平均薪水却较高，表明这些城市可能存在高薪职位。

(3) 薪酬待遇交互式轮播箱线图绘制

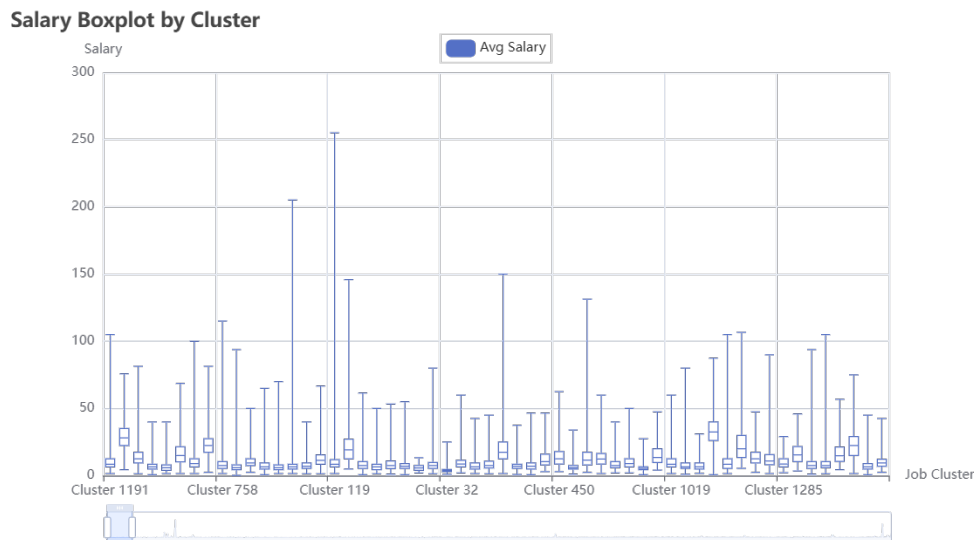


图 19: 薪酬待遇交互式轮播箱线图绘制

图 19展示了不同聚类 (Cluster) 的薪资分布情况，使用箱线图 (Boxplot) 进行可视化。通过箱线图，可以清晰地看到每个聚类的薪资中位数、四分位数、最大值和最小值，从而分析薪资待遇的分布特征和差异。不同聚类之间的薪资水平差异显著，部分聚类如 Cluster 119 显著高于其他聚类。大部分聚类的薪资水平较为接近，但存在少数高薪聚类，显示出职位市场的多样性。

3.2.5 结果分析

通过对三幅图（关键技能词云图、城市偏好双 Y 轴柱状-折线图、薪酬待遇箱线图）的分析，我们可以得出以下结论：

- **关键技能需求集中：**词云图显示出当前职位市场中最常见的技能需求。图中显示的关键词如 EdD、GP、GI 和 Eas 等频率较高，表示这些技能在职位中被广泛需求。
- **偏好城市：**职位数量较多的城市一般集中在前几个城市，这些城市可能是经济或产业中心，职位需求较大。随着城市数量的增加，职位数量迅速下降，显示出职位的地域集中现象。同时，一般情况下，职位数量较多的相较于职位数量少的城市薪酬相对高一点。
- **薪资集中度：**大多数职位的薪资分布集中在较低的范围，表明职位市场中大部分职位的薪资水平较为集中和稳定。

3.3 问题三：建模并识别薪酬模式和潜在的薪酬差异

在处理本问题时，将本问题划分为分类与回归两个子任务。为了探寻薪酬模式，将薪酬模型分为“按月分配”，“按周分配”，“按天分配”，“按小时分配”以及“按单分配”并将其分别量化为标签“0”，“1”，“2”，“3”，“4”用于分类任务；利用平均薪酬来进行回归预测。首先对原始数据进行特征

相关性分析，并进行特征构建。之后利用随机森林模型分别进行分类与回归任务，流程图如图 20 所示：

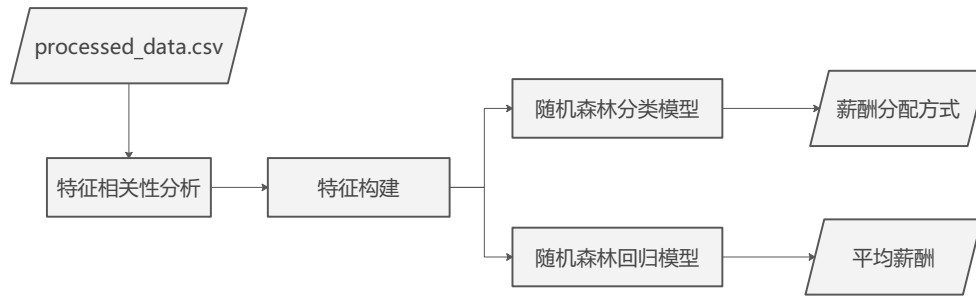


图 20: 问题三流程图

3.3.1 特征相关性分析与构建

分别绘制在不同的 city、job_title、company_type、experience 以及 education 下的平均薪酬箱线图用于判断特征的相关性，箱线图如图 21 所示：

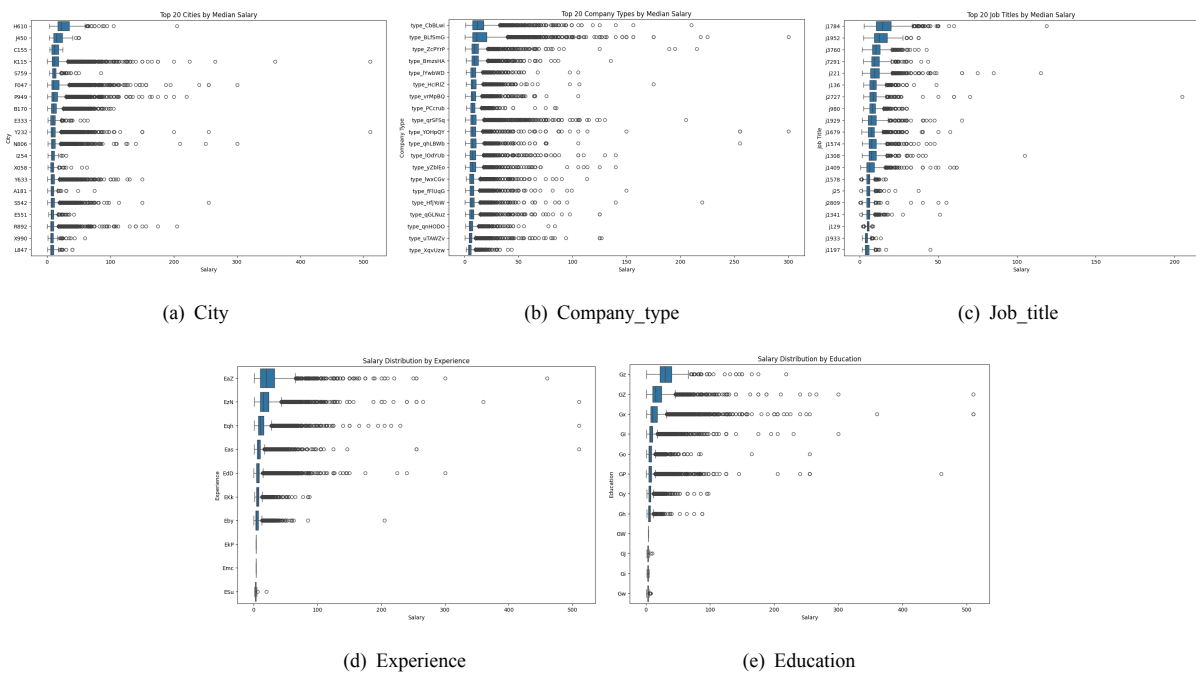


图 21: 特征相关性分析

由图 21 可以看出，city、job_title、company_type、experience 以及 education 特征对于薪酬的分布均有显著性影响。因此选择这五个特征作为职位的特征向量，用于后续的分类与回归任务。

在进行分类与回归任务之间，需要对上述五个特征进行量化处理，将每个特征赋予一个唯一的整数，不增加额外的特征维度。这种方法通常在基于树的模型中表现不错。

3.3.2 随机森林建模

将原始数据按训练集: 测试集 = 4:1 的比例随机划分，使用随机森林回归模型和随机森林分类模型分别进行薪资预测和薪资类别分类，并对模型的预测效果进行评估。详细的算法思路如下：

(1) 导入必要的库

导入 RandomForestRegressor 和 RandomForestClassifier 用于构建回归和分类模型。导入 mean_squared_error 和 accuracy_score 用于模型评估。

(2) 回归模型训练

实例化随机森林回归模型，设置 n_estimators=100 表示使用 100 棵决策树，random_state=42 保证结果可重复。使用训练数据 X_train 和 y_train['avg_salary'] 训练回归模型。核心 Python 代码如下：

```
1 # 回归模型
2 regressor = RandomForestRegressor(n_estimators=100, random_state=42)
3 regressor.fit(X_train, y_train['avg_salary'])
```

(3) 分类模型训练

实例化随机森林分类模型，设置 n_estimators=100 表示使用 100 棵决策树，random_state=42 保证结果可重复。使用训练数据 X_train 和 y_train['salary_category'] 训练分类模型。核心 Python 代码如下：

```
1 # 分类模型
2 classifier = RandomForestClassifier(n_estimators=100, random_state=42)
3 classifier.fit(X_train, y_train['salary_category'])
```

(4) 进行预测

使用训练好的回归模型对测试数据 X_test 进行薪资预测，得到预测结果 salary_pred。使用训练好的分类模型对测试数据 X_test 进行薪资类别预测，得到预测结果 category_pred。核心 Python 代码如下：

```
1 # 预测
2 salary_pred = regressor.predict(X_test)
3 category_pred = classifier.predict(X_test)
```

(5) 评估指标

1) 回归模型评估：使用 mean_squared_error 计算预测结果 salary_pred 与真实值 y_test['avg_salary'] 之间的均方误差（MSE），评估回归模型的误差。MSE 值越小，模型的预测效果越好。

2) 分类模型评估：使用 accuracy_score 计算预测结果 category_pred 与真实值 y_test['salary_category'] 之间的分类准确率。准确率越高，模型的分类效果越好。

3.3.3 可视化结果

(1) 评估指标

所建模型在回归方面的均方误差为 48.34，在分类方面的准确率为 98.99%。如图 22 所示，一定程度上佐证了所建模型的有效性。

```
Regression MSE: 48.33583514235542
Classification Accuracy: 0.9899020416700559
```

图 22: 评估指标

(2) 混淆矩阵

模型对于测试集上薪酬模式分类预测的混淆矩阵以及分类过程中特征的重要程度如图 23 所示：

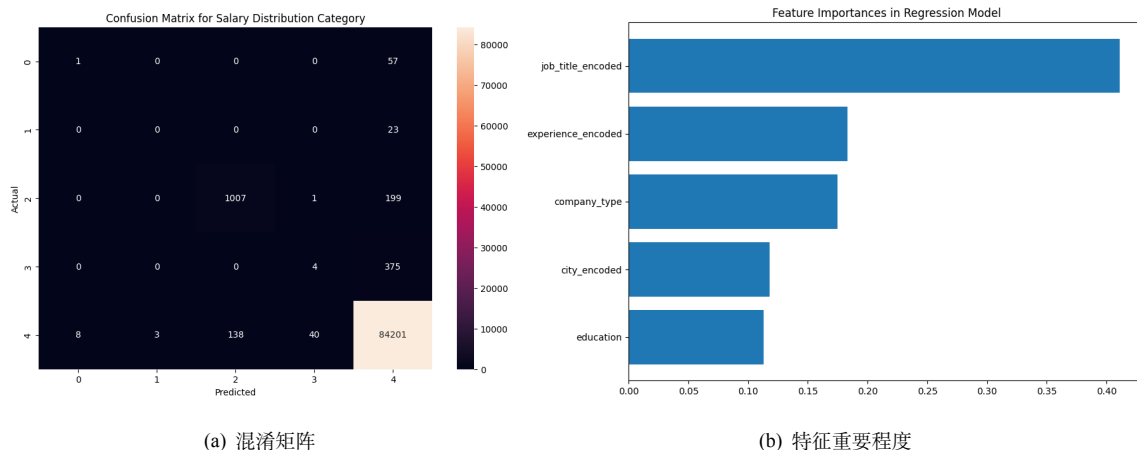


图 23: 分类结果

在混淆矩阵中，横轴表示预测的薪资类别（Predicted），纵轴为实际的薪资类别（Actual）。每个格子中的数值表示实际类别和预测类别的数量，颜色从浅到深表示数量从少到多。观察图 23 子图 (a) 可以看出模型有较好的分配预测效果。观察图 23 子图 (b) 可以看出职位名称、工作经验和公司类型是薪资预测的主要影响因素。

(3) 回归曲线

在横坐标为真实平均薪水，纵坐标为预测平均薪水的坐标系下绘制的回归效果曲线如图 24 所示：



图 24: 回归曲线

图 24 中，蓝色点大多分布在虚线附近，表示模型的预测值与实际值接近，说明回归模型有较好的预测效果。

3.3.4 结果分析

图 22、图 23 以及图 24 均表示，所构建的随机森林模型具有较好的分类与预测效果。

3.4 问题四：挖掘和总结地域招聘活动画像

3.4.1 偏好职位与薪水分布交互式轮播柱状-折线图绘制

1. 全局：各个城市的职位需求量柱状图及薪资水平折线图

利用 `pyecharts`，并融合柱状图与折线图来展示每个城市的职位总需求数量和城市的最高、最低及平均职位薪资统计数据。

计算每个城市的职位总需求数量和对应城市的最高、最低及平均职位薪资，将这些信息合并到各自的数据框中，然后通过条形图显示每个城市的职位总需求数量，通过折线图显示每个城市中的最高、最低及平均职位薪资，最终将两者结合在一起进行可视化展示。具体实现过程如下：

(1) 计算每个城市中的职位总需求数量：使用 `value_counts()` 方法统计每个城市的职位总需求数量，并按城市名称排序（默认是按职位总数量高低进行排序）。将结果重命名为 `city` 和 `city_job_counts` 列。

(2) 计算每个城市的平均职位工资、最高职位工资、最低职位工资：使用 `groupby()` 方法按城市分组，并计算平均工资、最高工资和最低工资。将结果重命名为 `city` 和 `avg_salary`, `max_salary`, `min_salary` 列。

(3) 创建薪资统计的折线图与职位数量的柱状图：使用 `pyecharts` 中的 `Line` 类创建折线图，显示每个城市的最低工资、最高工资和平均工资。使用 `pyecharts` 中的 `Bar` 类创建柱状图，显示每个城市的职位总需求数量。设置柱状图的透明度和颜色等配置项。

(4) 将折线图和柱状图组合：使用 `bar.overlap(line)` 方法将折线图和柱状图组合在一起。并设置图表的标题、坐标轴标签、轮播选项和图例位置。

核心代码详见附录，全局交互式轮播柱状-折线图预览如图 25 所示：

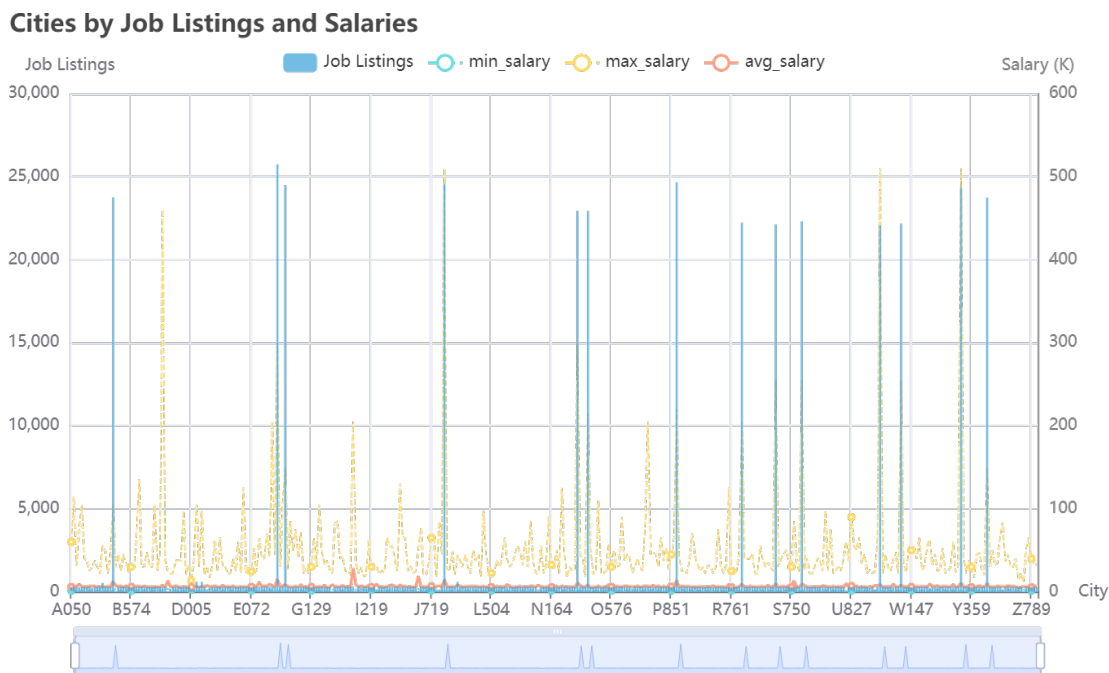


图 25: 全局城市交互式轮播柱状-折线图绘制预览-字母顺序

从图 25 中可以看出，各个城市的招聘活动地域分布差距大。

下面为了更直观的观察招聘活动主要集中在哪些城市，对横坐标按职位总数量高低进行排序操作，显示如图 26 所示：

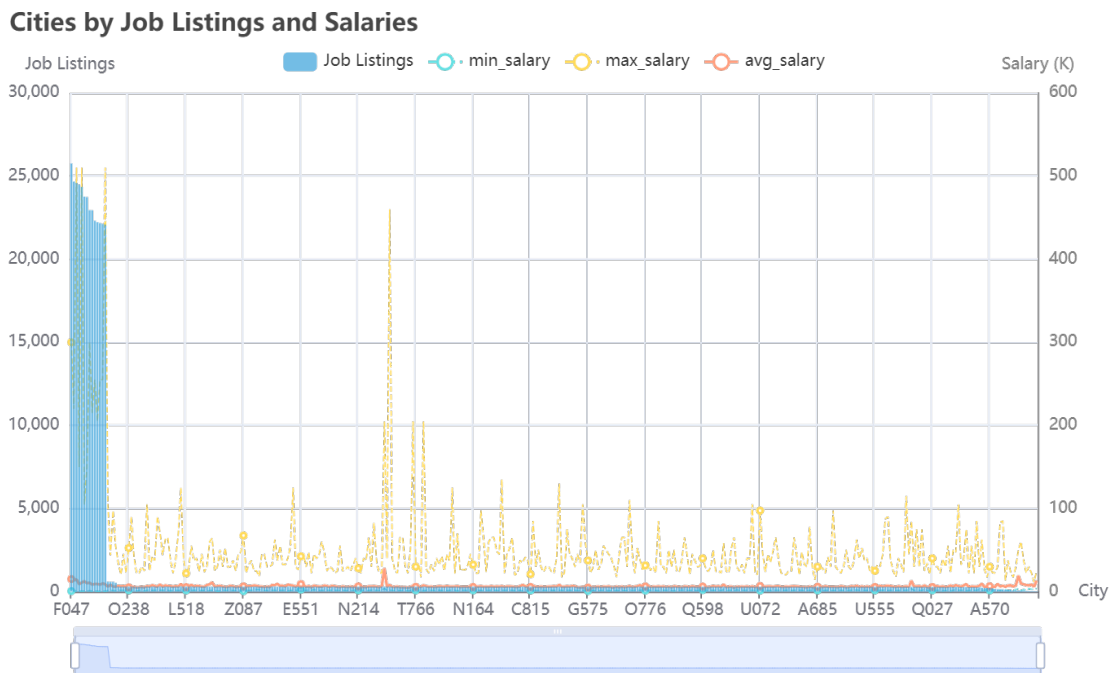


图 26: 全局城市交互式轮播柱状-折线图绘制预览-数量高低顺序

从图 26 可以看出，招聘活动主要集中在诸如 F047（总需求：25750）、P949（总需求：24664）、K115（总需求：24580）等这些城市。并且这些职位总需求量较大的城市普遍呈现具备较高的平均职位工资、最高职位工资、最低职位工资的现象。

2. 局部：单个城市的各个职位需求量柱状图及薪资水平折线图

利用 pyecharts，并融合柱状图与折线图来展示具体一个城市的各个职位需求数量和该职位的最高、最低及平均薪资统计数据。

具体计算一个城市的各个职位需求数量和该职位的最高、最低及平均薪资，将这些信息合并到各自的数据框中，然后通过条形图显示每个城市的职位总需求数量，通过折线图显示每个城市中的最高、最低及平均职位薪资，最终将两者结合在一起进行可视化展示。具体实现过程如下：

- (1) 定义要分析的城市（可以根据需要更改为任意城市）：使用 pandas 筛选出指定城市的数据。
- (2) 计算该城市的各个职位需求数量：使用 value_counts() 方法统计每个职位的需求数量，默认按职位需求数量高低进行排序。将结果重命名为 job_title 和 job_counts 列。
- (3) 计算每种职位的最高、最低和平均薪资：使用 groupby() 方法按职位分组，并计算平均工资、最高工资和最低工资。使用 reindex() 方法按职位需求量的顺序排列。将结果重命名为 job_title 和 avg_salary, max_salary, min_salary 列。
- (4) 创建薪资统计的折线图与职位需求量的柱状图：使用 pyecharts 中的 Line 类创建折线图，显示每个职位的最低工资、最高工资和平均工资。使用 pyecharts 中的 Bar 类创建柱状图，显示每个职位的总需求数量。设置柱状图的透明度和颜色、标记最大值和最小值的点等配置项。
- (5) 将折线图和柱状图组合：使用 bar.overlap(line) 方法将折线图和柱状图组合在一起。并设置图表的标题、坐标轴标签、轮播选项和图例位置。

核心代码详见附录，局部单个城市交互式轮播柱状-折线图预览如图 27 所示：

Job Listings in K115 by Job Title

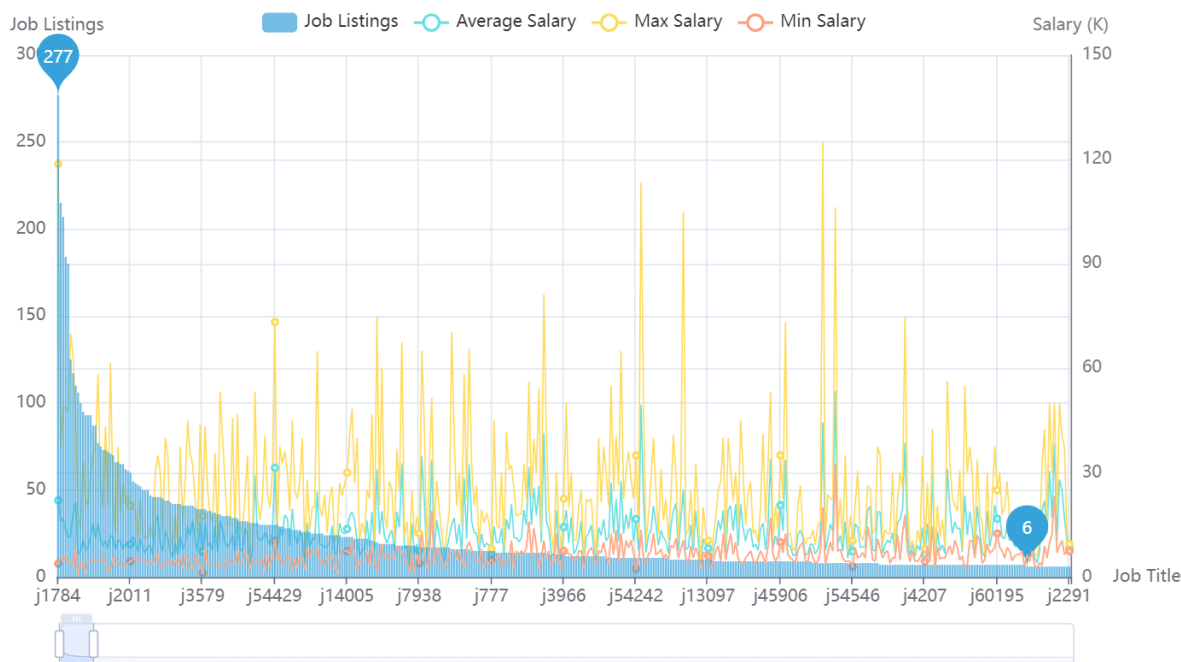


图 27: 城市 K115 交互式轮播柱状-折线图绘制预览

从图 27 中可以看出，K115 城市对各类职业的偏好差距大，其中，对 j1784 有 277 的需求量，对 j1952 有 215 的需求量，而对 j57927 和 j127902 等大多数职业只有 1 个需求量。

下面为了更直观的观察各个城市对职业的偏好情况，将所有城市的职业需求分布情况按 10×10 进行全局摆放，由于空间有限这里只展示部分，显示如图 28 所示：

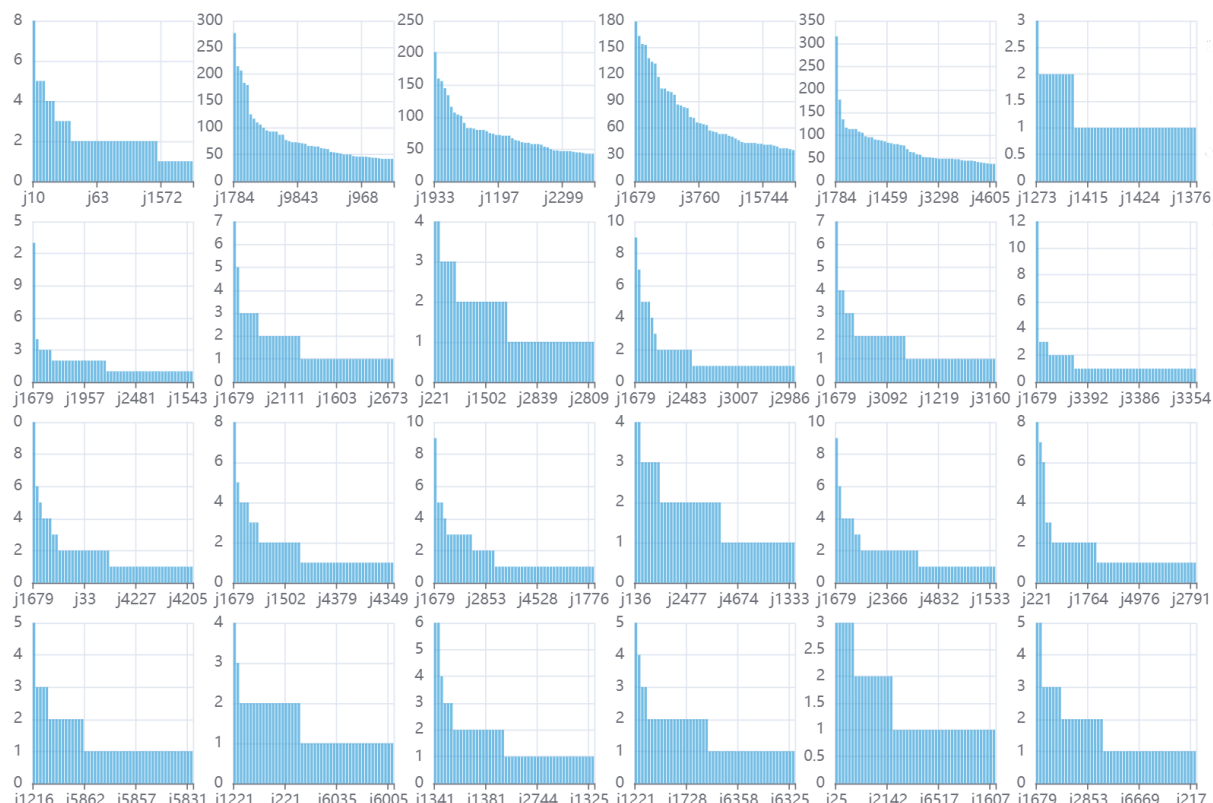


图 28: 各个城市对职业的偏好情况预览

从图 28 可以看出，大部分城市偏爱诸如就 j1679 类职业，具体的需要结合图像进行详细对照。

3.4.2 行业类别与薪水分布交互式轮播堆叠柱状-折线图绘制

1. 全局：各个城市的行业类别分布堆叠柱状图及折线图

利用 pyecharts，并融合堆叠柱状图与折线图来展示每个城市的行业类别分布和总行业类别数量等统计数据。

计算每个城市的各行业类别职位数量和对应城市的唯一行业类别数量，将这些信息合并到各自的数据框中，然后通过堆叠柱状图显示每个城市的各行业类别总职位需求数量分布，通过折线图显示每个城市中的唯一行业类别数量，最终将两者结合在一起进行可视化展示。具体实现过程如下：

(1) 按城市和行业类别统计职位数量，生成一个透视表 (pivot table)。计算每个城市的唯一行业类别数量。

(2) 创建计算每个城市的唯一行业类别数量的折线图与行业类别分布的柱状图：使用 pyecharts 中的 Line 类创建折线图，显示每个城市的唯一行业类别数量。使用 pyecharts 中的 Bar 类创建迭代柱状图，遍历每个行业类别，并添加相应的职位需求量数据到柱状图中，设置为堆叠显示。设置柱状图的透明度和颜色等配置项。

(3) 将折线图和堆叠柱状图组合：使用 bar.overlap(line) 方法将折线图和堆叠柱状图组合在一起。并设置图表的标题、坐标轴标签、轮播选项和图例位置。

核心代码详见附录，全局交互式轮播堆叠柱状-折线图预览如图 29 所示：

Industry Distribution by City

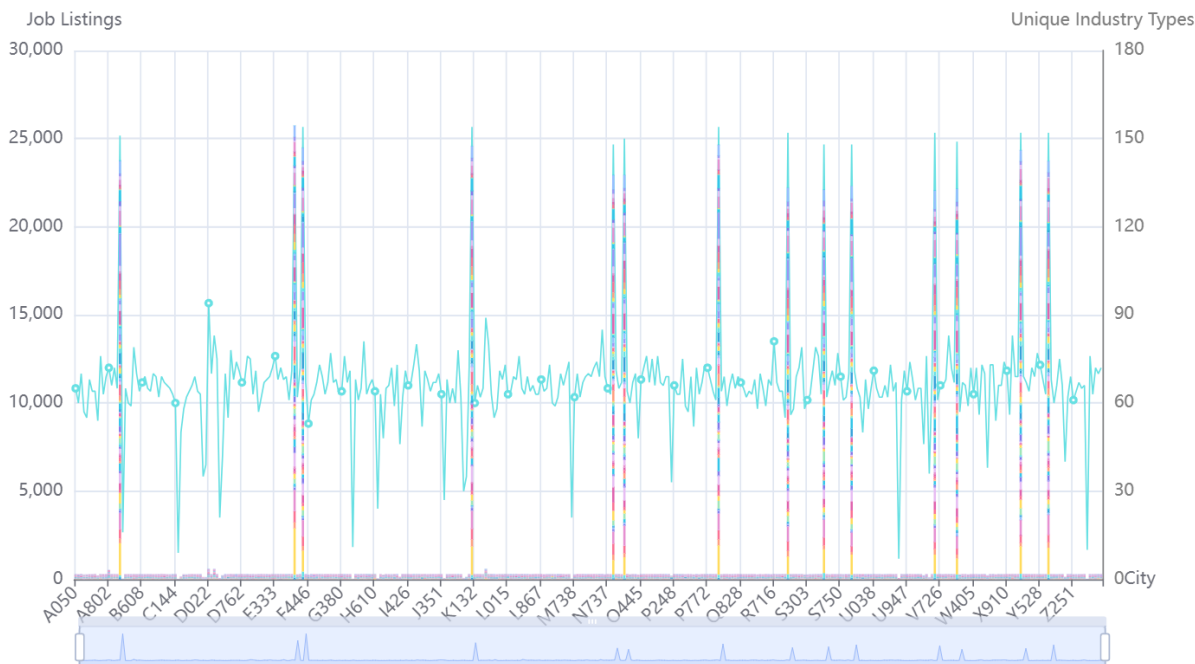


图 29: 全局城市行业类别分布交互式轮播堆叠柱状-折线图绘制预览

从图 29 中可以看出，各个城市的招聘活动行业类别分布差距大。

2. 局部：单个城市的行业类别分布柱状图及薪资水平折线图

利用 `pyecharts`，并融合柱状图与折线图来展示具体一个城市的各个行业类别的职位需求数量和该行业类别的最高、最低及平均职位薪资统计数据。

具体计算一个城市的各个行业类别分布数量和该行业类别的最高、最低及平均职位薪资，将这些信息合并到各自的数据框中，然后通过条形图显示每个城市的各个行业类别的职位总需求数量，通过折线图显示每个行业类别中的最高、最低及平均职位薪资，最终将两者结合在一起进行可视化展示。具体实现过程如下：

- (1) 定义要分析的城市（可以根据需要更改为任意城市）：使用 `pandas` 筛选出指定城市的数据。
- (2) 统计该城市各行业类别的职位需求量：使用 `value_counts()` 方法统计每个行业类别的职位需求数量，默认按职位需求数量高低进行排序。将结果重命名为 `company_type` 和 `job_counts` 列。
- (3) 计算每种行业类别的最高、最低和平均职位薪资：使用 `groupby()` 方法按行业类别分组，并计算平均工资、最高工资和最低工资。使用 `reindex()` 方法按各个行业类别的职位需求量的顺序排列。将结果重命名为 `company_type` 和 `avg_salary, max_salary, min_salary` 列。
- (4) 创建薪资统计的折线图与各个行业类别的职位需求量的柱状图：使用 `pyecharts` 中的 `Line` 类创建折线图，显示每个职位的最低工资、最高工资和平均工资。使用 `pyecharts` 中的 `Bar` 类创建柱状图，显示每个行业类别下的职位总需求数量。设置柱状图的透明度和颜色、标记最大值和最小值的点等配置项。
- (5) 将折线图和柱状图组合：使用 `bar.overlap(line)` 方法将折线图和柱状图组合在一起。并设置图表的标题、坐标轴标签、轮播选项和图例位置。

核心代码详见附录，局部单个城市交互式轮播柱状-折线图预览如图 30 所示：

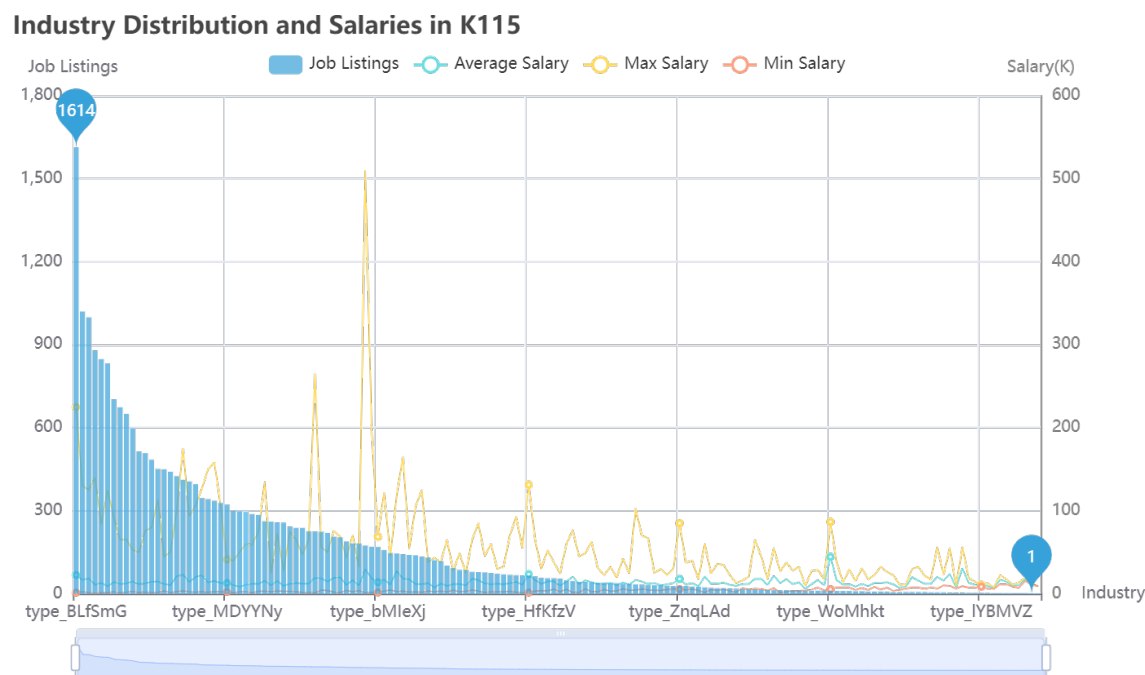


图 30: 城市 K115 行业类别及薪水分布交互式轮播柱状-折线图绘制预览

从图 30 中可以看出，K115 城市对各行业类别的分布差距大，其中，对 type_BLfSmG 行业领域有 1614 个职位需求量，对 type_qrSFSq 行业领域有 1020 的需求量，而对 ype_tjGjDL 和 type_CiMmHM 等大多数行业领域只有 1 个职位需求量。

3.4.3 K-means 聚类分析

(1) 选择特征列

使用 pivot_table 方法分别对城市和职位类别、城市和行业类别进行聚合统计，生成职位需求量和行业需求量的特征矩阵。

(2) 合并特征

将职位需求量和行业需求量的特征矩阵进行合并，生成综合特征矩阵。合并后的部分属性展示如图 31 所示。

	j1	j10	j100	j1000	j10000	j100000	j100001	j100002	j100003	j100004	...	type_wiXJCA	type_xKVDuy	type_xfBgF	type_xjtctu	type_xkGm
city																
A050	0	0	0	0	0	0	0	0	0	0	...	0	1	3	1	
A149	0	0	0	0	0	0	0	0	0	0	...	1	0	3	0	
A155	0	0	0	0	0	0	0	0	0	0	...	0	1	7	0	
A181	0	0	0	0	0	0	0	0	0	0	...	2	0	2	1	
A195	0	0	0	0	0	0	0	0	0	0	...	0	4	5	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
Z581	0	0	0	0	0	0	0	0	0	0	...	0	4	1	0	
Z731	0	0	0	0	0	0	0	0	0	0	...	0	0	4	0	
Z789	0	1	0	0	0	0	0	0	0	0	...	0	6	6	1	
Z824	0	0	0	0	0	0	0	0	0	0	...	0	1	4	2	
Z885	0	0	0	0	0	0	0	0	0	0	...	1	1	5	3	

371 rows × 169692 columns

图 31: 特征列选择、合并、量化

(3) 标准化数据

使用 StandardScaler 对特征矩阵进行标准化处理。标准化后如图 32 所示。

```
array([[ -0.05198752, -0.14822033, -0.09028939, ..., -0.18168814,
        -0.10249798, -0.20554681],
       [ -0.05198752, -0.14822033, -0.09028939, ..., -0.20378374,
        -0.10249798, -0.20554681],
       [ -0.05198752, -0.14822033, -0.09028939, ..., -0.22587934,
        -0.10249798, -0.20554681],
       ...,
       [ -0.05198752, -0.02092926, -0.09028939, ..., -0.21114894,
        -0.10249798, -0.20554681],
       [ -0.05198752, -0.14822033, -0.09028939, ..., -0.17432294,
        -0.10249798, -0.20554681],
       [ -0.05198752, -0.14822033, -0.09028939, ..., -0.21851414,
        -0.10249798, -0.20554681]])
```

图 32: 标准化数据

(4) 使用肘部法确定最佳聚类数

```
1 inertia = []
2 for n in range(1, 30):
3     kmeans = KMeans(n_clusters=n, n_init=10, random_state=42)
4     kmeans.fit(scaled_features)
5     inertia.append(kmeans.inertia_)
```

代码所绘制的肘部图如图 33 所示：

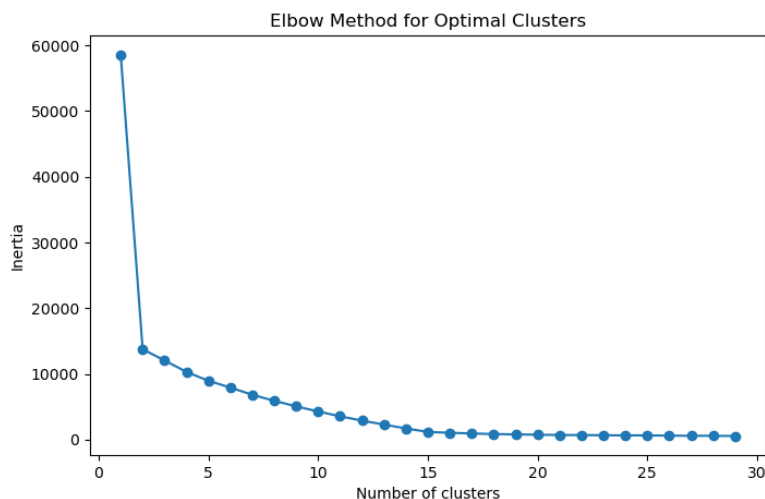


图 33: 肘部图

观察图 33 可以看出，“肘部”即折线斜率变化较大的位置在聚类数目为 3 处，因此，**最佳聚类数目为 3**。故取 $K=3$ 时，重新进行 K-means 聚类，为每个 company_type 添加一个 cluster 标签以便后续可视化分析。

这里我们分别进行了对所有职位和行业类别进行特征向量化、对所有职位进行特征向量化和对所有行业类别进行特征向量化后分别进行聚类，最终效果较佳的是对所有行业类别进行特征向量化后进行聚类，所以这里只展示此处理下的 PCA 降维效果。

3.4.4 PCA 降维

使用 PCA 进行降维，将数据从高维空间投影到二维空间。

```
1 pca = PCA(n_components=2)
2 pca_features = pca.fit_transform(scaled_features)
3 pca_df = pd.DataFrame(pca_features, columns=['PC1', 'PC2'])
4 pca_df['Cluster'] = city_clusters
5 plt.figure(figsize=(10, 7))
6 sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster', palette='viridis')
7 plt.title('PCA of Clustering Results')
8 plt.show()
```

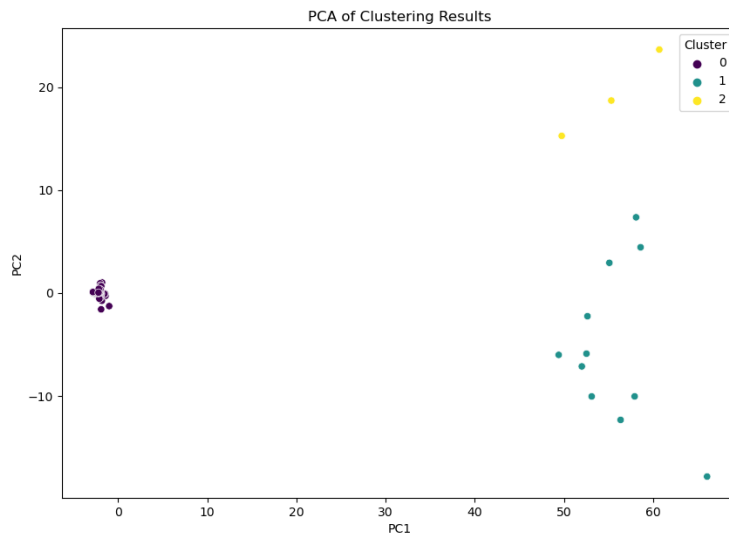


图 34: PCA 降维可视化

3.4.5 可视化结果

(1) 偏好职位与薪水分布交互式轮播柱状-折线图

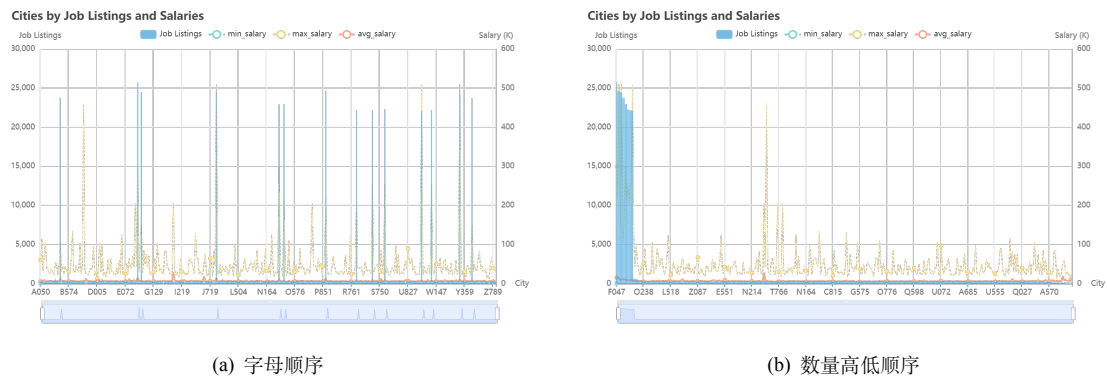


图 35: 全局城市职位总分布交互式轮播柱状-折线图

从图 35 可以看出，招聘活动主要集中在诸如 F047（总需求：25750）、P949（总需求：24664）、

K115（总需求：24580）等这些大城市。并且这些职位总需求量较大的城市普遍呈现具备较高的平均职位工资、最高职位工资、最低职位工资的现象。（2）行业类别与薪水分布交互式轮播堆叠柱状-折线图

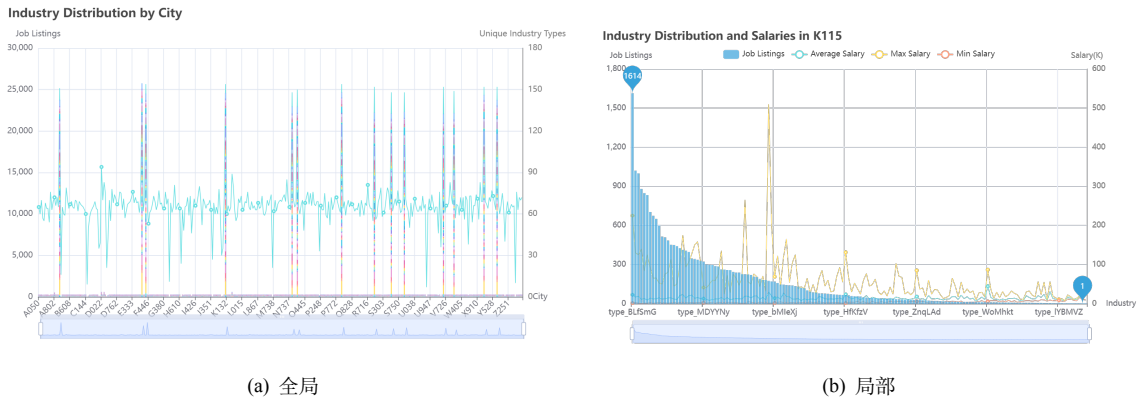


图 36: 行业类别与薪水分布交互式轮播堆叠柱状-折线图

从图 36 可以看出,各个城市的招聘活动行业类别分布差距大,其中 type_BLfSmG 和 type_CbBLwi 等行业领域在各城市具有不小的比例,但大部分行业领域仅出现在个别的城市中。其中, **K115 城市对各行业类别的分布差距大**,该城市对 type_BLfSmG 行业领域有 1614 个职位需求量,对 type_qrSFSq 行业领域有 1020 的需求量,而对 ype_tjGjDL 和 type_CiMmHM 等大多数行业领域只有 1 个职位需求量。

通过选择不同的特征列进行向量聚类,最终采取效果较佳的,对所有行业类别进行特征向量化后进行聚类,最终的 PCA 降维效果如图 37。

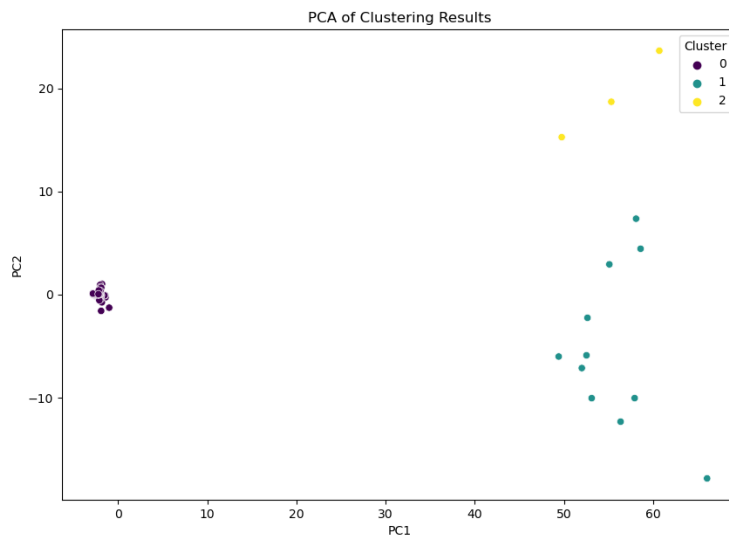


图 37: PCA 降维可视化

3.4.6 结果分析

通过上面可视化图的分析,我们可以得出以下结论:

- **城市职位需求量：**在分析的各个城市中，职位需求量差异明显，主要集中在一些经济发达、人口密集的城市。例如，F047、P949、K115 等城市的职位需求量明显高于其他城市。
- **薪资水平：**职位需求量较大的城市普遍拥有较高的平均职位工资、最高职位工资和最低职位工资。这表明这些城市的招聘活动不仅频繁，而且薪资水平也具有竞争力。整体来看，职位需求量和薪资水平之间呈现正相关关系，即职位需求量越大的城市，其薪资水平也越高。
- **行业类别分布：**各个城市的行业类别分布存在明显差异。某些城市的招聘活动集中在特定的行业，而另一些城市则具有更为多样化的行业需求。
- **行业多样性：**通过计算每个城市的唯一行业类别数量，可以看出一些城市的行业多样性较高，表明这些城市具有更广泛的经济活动和更丰富的就业机会。
- **聚类效果：**通过肘部法确定最佳聚类数为 3，将城市划分为 3 个不同的招聘特征聚类。不同聚类之间的招聘活动特征差异明显。
- **聚类效果：**在降维后的 PCA 图中，不同聚类的城市分布较为清晰，表明这些城市在招聘活动方面具有明显的特征差异。例如，某些聚类的城市集中在高需求高薪资的职位，而另一些聚类则集中在特定行业的招聘活动中。

3.5 问题五：行业发展动态与新兴职位识别

结合上述分析结果，我们可以总结行业发展动态，并识别出市场急需人才的新兴职位。

3.5.1 行业发展动态

- **行业集中性与高薪职位的关系：**分析显示，某些行业类别的职位需求量集中，且这些行业往往提供更高的薪资。这反映了行业专业化和技术要求的高度集中，高薪职位往往集中在需求技能高、专业性强的行业。
- **行业多样性与地域差异：**同城市的行业需求和薪资水平展示了明显的地域差异。一些城市特别是大城市，由于经济和工业发展水平较高，所以拥有更多高薪和多样化的职位机会。

3.5.2 急需人才的新兴职位

- **技术和管理领域的新兴职位：**从高薪职位的需求量来看，技术和管理领域的职位需求量将会明显增加。特别是在技术快速发展的今天，如数据分析师、软件开发师等专业技术职位需求大增，这些职位通常要求高级教育背景和专业技能。
- **创新和研发职位：**随着行业向高新技术和可持续发展转型，相关的研发和创新类职位需求量也在增加。这些职位如绿色能源工程师、生物技术研究员等，不仅反映了行业发展的新方向，也是高薪和急需的新兴职位。

3.5.3 理由与实际观察

- **薪资水平与职位需求的相关性：**薪通常与高需求和高专业技能直接相关。在经济全球化和技术快速发展的背景下，对于具备特定技术和能力的人才需求量大增。
- **市场与技术发展趋势：**市场需求是推动职位形成和发展的关键因素。例如，随着人工智能和大数据的广泛应用，相关职位的需求迅速增长。

- **教育与行业需求对接：**高等教育机构的课程和专业设置越来越多地反映市场需求，突出了教育与行业需求的紧密联系。

通过分析，我们可以识别出当前市场上急需人才的新兴职位，并了解行业的发展趋势。对于求职者和教育机构，这些信息非常宝贵，可以帮助他们做出更符合市场需求的职业和教育选择。对于企业而言，这有助于精确定位人才招聘策略，特别是在竞争激烈的高技能行业领域。

四、附录：核心代码

4.1 问题一核心代码

4.1.1 交互式轮播柱状-折线图绘制

```

1  # CREATE THE LINE CHART FOR SALARY STATISTICS
2  line = (
3      Line()
4      .add_xaxis(cluster_data['cluster'].tolist())
5      .add_yaxis(
6          "AvgSalary",
7          cluster_data['avg_salary_job_data'].tolist(),
8          yaxis_index=1,
9          label_opts=opts.LabelOpts(is_show=False),
10         linestyle_opts=opts.LineStyleOpts(type_="solid", width=2)
11     )
12 )
13 # CREATE THE BAR CHART FOR JOB AND COMPANY COUNTS
14 bar = (
15     Bar(init_opts=opts.InitOpts(theme=ThemeType.LIGHT))
16     .add_xaxis(cluster_data['cluster'].tolist())
17     .add_yaxis(
18         "Company_type_Count",
19         cluster_data['company_count'].tolist(),
20         markline_opts=opts.MarkLineOpts(
21             data=[opts.MarkLineItem(type_="average", name="AverageCompanyTypeCount")],
22             linestyle_opts=opts.LineStyleOpts(type_="dashed", color="red")
23         ),
24         yaxis_index=1,
25         label_opts=opts.LabelOpts(is_show=False, position="top"),
26         itemstyle_opts=opts.ItemStyleOpts(opacity=0.6, color="pink")
27     )
28     .extend_axis(
29         yaxis=opts.AxisOpts(
30             name="Salary(K)",
31             type_="value",
32             min_=0,
33             max_=cluster_data['avg_salary_job_data'].max() * 1.1, # 增加一些空白以确保完全显示
34             position="right",
35             axisline_opts=opts.AxisLineOpts(
36                 linestyle_opts=opts.LineStyleOpts(color="gray")
37             ),
38             axislabel_opts=opts.LabelOpts(formatter="{value}")
39         )
40     )
41     .set_global_opts(
42         title_opts=opts.TitleOpts(title="CompanyCountbyClusterwithSalaryStats"),
43         xaxis_opts=opts.AxisOpts(name="JobCluster"),

```

```

44     yaxis_opts=opts.AxisOpts(name="Company_Count", min_=0, max_=cluster_data['company_count'].max()), # 调整左侧纵坐标轴范围
45     datazoom_opts=opts.DataZoomOpts(), # ADD ZOOM CAPABILITY
46     legend_opts=opts.LegendOpts(pos_top="5%") # ADJUST LEGEND POSITION
47 )
48 )
49 # COMBINE THE BAR AND LINE CHARTS
50 bar.overlap(line)
51 # RENDER THE CHART TO AN HTML FILE AND DISPLAY
52 bar.render_notebook()

```

Code Listing 1: 利用 Pyecharts 绘制交互式轮播柱状-折线图

4.1.2 交互式轮播柱状-折线图绘制

```

1  # CREATE A STACKED BAR CHART FOR JOB COUNTS
2  bar = (
3      Bar(init_opts=opts.InitOpts(theme=ThemeType.LIGHT))
4      .add_xaxis(cluster_data['cluster'].astype(str).tolist())
5  )
6
7  # ADD STACKED DATA FOR EACH EXPERIENCE LEVEL
8  for experience in experience_distribution.columns:
9      bar.add_yaxis(
10         experience,
11         experience_distribution[experience].tolist(),
12         stack="stack1",
13         label_opts=opts.LabelOpts(is_show=False)
14     )
15
16  bar.set_global_opts(
17      title_opts=opts.TitleOpts(title="Experience_Requirements_by_Job_Cluster"),
18      xaxis_opts=opts.AxisOpts(name="Job_Cluster"),
19      yaxis_opts=opts.AxisOpts(name="Job_Count", min_=0),
20      datazoom_opts=opts.DataZoomOpts(), # ADD ZOOM CAPABILITY
21      legend_opts=opts.LegendOpts(pos_top="5%") # ADJUST LEGEND POSITION
22  )
23
24  # RENDER THE CHART
25  bar.render_notebook()

```

Code Listing 2: 利用 Pyecharts 绘制交互式轮播柱状-折线图

4.2 问题二核心代码

4.2.1 词云图绘制

```

1  # 创建词云图
2  wordcloud = (
3      WordCloud()
4      .add("", data, word_size_range=[20, 100], shape="cardioid")
5      .set_global_opts(
6          title_opts=opts.TitleOpts(
7              title=f"Key_Skills_For_Jobs",
8              title_textstyle_opts=opts.TextStyleOpts(font_size=23)
9          ),

```

```

10     tooltip_opts=opts.TooltipOpts(is_show=True),
11 )
12 .set_series_opts(
13     label_opts=opts.LabelOpts(is_show=False),
14     textstyle_opts=opts.TextStyleOpts(font_family="cursive", font_weight="bold")
15 )
16 )
17 wordcloud.render_notebook()

```

Code Listing 3: 利用 Pyechart 绘制词云图

4.2.2 交互式轮播堆叠柱状图绘制

```

1 # 添加X轴标签和Y轴数据
2 boxplot.add_xaxis(cluster_labels)
3 boxplot.add_yaxis("Avg_Salary", prepared_data)
4 # 设置全局选项
5 boxplot.set_global_opts(
6     title_opts=opts.TitleOpts(title="Salary_Boxplot_by_Cluster"),
7     legend_opts=opts.LegendOpts(pos_top="5%"),
8     xaxis_opts=opts.AxisOpts(name="Job_Cluster"),
9     datazoom_opts=opts.DataZoomOpts(),
10    yaxis_opts=opts.AxisOpts(name="Salary")
11 )
12 # 渲染图表
13 # BOXPLOT.RENDER_NOTEBOOK()
14 boxplot.render("./Result/Q2-3薪酬待遇交互式轮播箱线图绘制.html")

```

Code Listing 4: 利用 Pyecharts 绘制交互式轮播堆叠柱状图

4.3 问题三核心代码

```

1 from sklearn.metrics import confusion_matrix
2 import seaborn as sns
3
4 sns.heatmap(cm, annot=True, fmt='d')

```

Code Listing 5: 混淆矩阵绘制

4.4 问题四核心代码

4.4.1 交互式轮播柱状-折线图绘制

```

1 # 创建薪资统计的折线图
2 line = (
3     Line()
4     .add_xaxis(city_job_counts.index.tolist())
5     .add_yaxis(
6         "min_salary",
7         city_salary_stats['min_salary'].tolist(),
8         yaxis_index=1,
9         label_opts=opts.LabelOpts(is_show=False),
10        linestyle_opts=opts.LineStyleOpts(type_="dashed")
11    )
12    .add_yaxis(

```

```

13     "max_salary",
14     city_salary_stats['max_salary'].tolist(),
15     yaxis_index=1,
16     label_opts=opts.LabelOpts(is_show=False),
17     linestyle_opts=opts.LineStyleOpts(type_="dashed"),
18 )
19 .add_yaxis(
20     "avg_salary",
21     city_salary_stats['avg_salary'].tolist(),
22     yaxis_index=1,
23     label_opts=opts.LabelOpts(is_show=False),
24     linestyle_opts=opts.LineStyleOpts(type_="solid", width=2)
25 )
26 )
27 # 创建柱状图
28 bar = (
29     Bar(init_opts=opts.InitOpts(theme=ThemeType.LIGHT))
30     .add_xaxis(city_job_counts.index.tolist())
31     .add_yaxis("Job_Listings", city_job_counts.values.tolist(),
32               label_opts=opts.LabelOpts(is_show=False),
33               itemstyle_opts=opts.ItemStyleOpts(opacity=0.7))
34     .extend_axis(
35         yaxis=opts.AxisOpts(
36             name="Salary_K",
37             type_="value",
38             min_=0,
39             position="right",
40             axisline_opts=opts.AxisLineOpts(
41                 linestyle_opts=opts.LineStyleOpts(color="gray")
42             ),
43             axislabel_opts=opts.LabelOpts(formatter="{value}")
44         )
45     )
46 )
47 # 组合柱状图和折线图
48 bar.overlap(line)

```

Code Listing 6: 利用 Pyecharts 绘制交互式轮播柱状-折线图

4.4.2 交互式轮播堆叠柱状-折线图绘制

```

1 # 创建堆叠柱状图
2 bar = Bar(init_opts=opts.InitOpts(theme=ThemeType.LIGHT))
3 bar.add_xaxis(city_industry_counts.index.tolist())
4 # 添加各行业类别的职位需求量数据，并堆叠
5 for industry in city_industry_counts.columns:
6     bar.add_yaxis(industry, city_industry_counts[industry].tolist(), stack="stack1", label_opts=opts.
7                   LabelOpts(is_show=False))
8 # 创建折线图，用于展示每个城市的行业类别数量
9 line = Line()
10 line.add_xaxis(city_industry_unique_counts.index.tolist())
11 line.add_yaxis(
12     "Unique_Industry_Types",
13     city_industry_unique_counts.tolist(),
14     yaxis_index=1,
15     label_opts=opts.LabelOpts(is_show=False),
16     linestyle_opts=opts.LineStyleOpts(width=1)

```

```
17 )
18 # 扩展Y轴，用于展示行业类别数量
19 bar.extend_axis(
20     yaxis=opts.AxisOpts(
21         name="Unique_Industry_Types",
22         type_="value",
23         position="right",
24         axisline_opts=opts.AxisLineOpts(
25             linestyle_opts=opts.LineStyleOpts(color="gray")
26         ),
27         axislabel_opts=opts.LabelOpts(formatter="{value}")
28     )
29 )
30 # 组合柱状图和折线图
31 bar.overlap(line)
```

Code Listing 7: 利用 Pyecharts 绘制交互式轮播柱状-折线图